

Universidad de La Habana
Facultad de Matemática y Computación

**Análisis comparativo de las metaheurísticas
IVNS y VNS para la resolución del
Problema de Enrutamiento de Vehículos
con Restricción de Capacidad**

Autora: Olivia Ibañez Mustelier

Tutores:

Lic. Alejandra Monzón Peña
MSc. Fernando Raúl Rodríguez Flores

Trabajo de Diploma para optar por el título de
Licenciada en Ciencia de la Computación

La Habana, junio de 2026

Dedicatoria

“

A mi familia, porque antes de aprender a caminar por esta carrera, aprendí a caminar junto a ustedes.

A mi madre, por vivir cada etapa junto a sus hijos y guiarnos siempre hacia lo que considera mejor.

A mi padre, por ayudarme cada vez que lo necesito y confiar en mí incluso cuando el camino parece difícil.

A mi hermano mayor, mi compañero y mi apoyo silencioso.

A mis tíos y a toda mi familia, por ser el lugar al que siempre puedo regresar.

Y a mi perro, mi pequeño sostén emocional, que, aunque no entiende de tesis, algoritmos ni desvelos, siempre sabe cuándo debe quedarse a mi lado.

Esta etapa también les pertenece. Yo llegué hasta aquí con mis pasos, pero con el amor de ustedes como camino.

”

Agradecimientos

Llegar al final de una etapa invita a mirar hacia atrás. Cuando lo hago, no veo un camino solitario. Veo muchas manos, muchas voces y muchas formas de cariño que me acompañaron hasta aquí.

Mi primer agradecimiento pertenece a mi familia, porque su presencia no depende de las circunstancias ni de la distancia.

A mi madre, que vive los estudios de sus hijos como si también fueran suyos, siente nuestros miedos, celebra cada pequeño avance y convierte cada preocupación en una forma de cuidado. Gracias por saber muchas veces qué necesitamos incluso antes de que nosotros mismos lo comprendamos. Gracias por aconsejarnos, aunque no siempre escuchemos en el momento, y por no cansarte de mostrarnos el camino que consideras mejor. Con el tiempo, casi siempre terminamos por reconocer que tenías razón.

A mi padre, porque quizá no siempre anuncie que todo saldrá bien, pero demuestra su confianza de una forma aún más firme. Cuando necesito algo, sin importar qué sea ni cuán difícil parezca, busca la manera de ayudarme. Gracias por darme cuanto está a tu alcance, por responder siempre cuando te necesito y por confiar en mí incluso cuando yo misma dudo.

A mi hermano mayor, mi compañero y mi apoyo silencioso. Muchas veces supo que algo no estaba bien aunque yo no dijera una palabra. Gracias por cuidarme, consentirme y permanecer cerca sin necesidad de preguntas. Tu manera de apoyar no siempre hace ruido, pero nunca pasa inadvertida.

A mis tíos, a los amigos de la familia y a todos los que hacen de mi hogar un refugio constante, gracias por verme crecer, celebrar mis logros y sostenerme en cada nueva etapa.

Agradezco de manera especial a mi primo, cuya ayuda me permitió conseguir la computadora que usé durante la carrera. Tal vez para otros solo sea una herramienta, pero para mí también guarda horas de estudio, errores, aprendizajes, experimentos, cansancio y sueños.

A esas amigas de la familia que, aun desde lejos, nunca permiten que la distancia se convierta en ausencia: gracias por encontrar siempre una manera de acompañarnos, aliviar algunas cargas y tender la mano cuando más falta hace. Su ayuda llega muchas

veces en silencio, pero deja una huella que siempre recordaremos.

A mis amigos del barrio y a quienes llegaron a mi vida antes de la universidad, gracias por permanecer. El tiempo cambia las rutinas y separa algunos caminos, pero conservar el afecto a través de los años constituye una de las formas más sinceras de amistad.

A mis amigos de la universidad, gracias por comprender sin exigir demasiadas explicaciones. Compartimos la misma carrera, los mismos desvelos, las mismas dudas y esa mezcla extraña de cansancio y esperanza que solo entiende quien atraviesa algo semejante. Al *Team Five*, mi círculo más cercano, gracias por las risas, la compañía, la confianza y por convertir los días difíciles en recuerdos que también merecen cariño. A las demás personas que encontré durante estos años, gracias por cada conversación, cada ayuda y cada momento que hizo más ligero el camino.

A mis tutores, gracias por confiar en mí y entregarme la responsabilidad de desarrollar este tema. A Ale, que forma parte de mi historia universitaria desde mis primeros pasos y a quien considero mi tutora desde mucho antes de que esta tesis comenzara. Gracias por enseñarme, orientarme y acompañar mi crecimiento durante la carrera. A Fernan, gracias por mostrarme que el conocimiento no termina en un aula y que el arte, la ciencia y la vida pueden encontrarse. Gracias por enseñarme a utilizar lo que sé, por acercar la teoría a la realidad y por despertar en mí el deseo de aprender más allá de una calificación.

A Celia, a Wilfredo y todos aquellos profesores que coincidieron conmigo durante las asignaturas y también a quienes nunca me dieron clases, pero compartieron conmigo la cultura, el deporte y la vida universitaria: gracias. La edad y los cargos nunca impidieron que naciera el cariño. En muchos casos, más que una relación entre estudiante y profesor, encontré una amistad que guardaré con afecto.

Tal vez después de estas páginas la vida nos lleve por caminos distintos y quizá algunos encuentros no vuelvan a repetirse. Aun así, haber coincidido con ustedes constituye una de las partes más valiosas de este recorrido.

Esta tesis lleva mi nombre, pero en sus páginas también viven la paciencia, las enseñanzas y la confianza de muchas personas. A todas ellas, gracias por verme crecer, por estar cerca mientras aprendía a superar mis propios límites y por formar parte de una etapa que hoy termina para abrir el camino hacia la siguiente.

Resumen

El Problema de Enrutamiento de Vehículos con Restricción de Capacidad (CVRP) busca construir rutas que atiendan a todos los clientes sin superar la capacidad de los vehículos y con el menor costo posible. Esta investigación compara la Búsqueda de Vecindad Variable (VNS) y la Búsqueda de Vecindad Infinitamente Variable (IVNS) en la resolución del CVRP. VNS recorre un conjunto finito de criterios de vecindad, mientras que IVNS usa una gramática libre de contexto para crear nuevos criterios durante la ejecución.

El estudio organiza la experimentación en tres etapas. La primera establece una referencia inicial entre VNS e IVNS. La segunda examina ajustes de generación, estrategias de búsqueda local y límites de tiempo por vecindad dentro de IVNS. La tercera compara con VNS la variante que la exploración elige mediante semillas nuevas y soluciones iniciales iguales dentro de cada par. Los experimentos usan casos de las familias **A** y **F** de CVRPLIB y analizan la brecha porcentual, el tiempo de ejecución y la estabilidad. El análisis estadístico aplica pruebas no paramétricas y la corrección de Holm.

En los casos de la familia **A**, la variante **baseline-first-t1.00** de IVNS alcanza menores brechas que VNS en la evaluación final, aunque VNS requiere menos tiempo en los casos de mayor tamaño. En **F-n135-k7**, VNS supera a IVNS tanto en calidad como en tiempo. Estos resultados muestran que el rendimiento de IVNS depende de la configuración y del caso de prueba. Por tanto, la creación de criterios durante la ejecución no garantiza por sí sola una ventaja frente a VNS.

Abstract

The Capacitated Vehicle Routing Problem (CVRP) seeks to construct routes that serve all customers without exceeding vehicle capacity and with the lowest possible cost. This research compares Variable Neighborhood Search (VNS) and Infinitely Variable Neighborhood Search (IVNS) for solving the CVRP. VNS explores a finite set of neighborhood structures, whereas IVNS uses a context-free grammar to create new neighborhood structures during execution.

The study organizes the experimentation into three stages. The first stage establishes an initial reference between VNS and IVNS. The second examines generation settings, local search strategies, and neighborhood time limits within IVNS. The third compares VNS with the variant chosen during the exploration stage by using new seeds and identical initial solutions within each pair. The experiments use instances from families **A** and **F** of CVRPLIB and analyze the percentage gap, execution time, and stability. The statistical analysis applies nonparametric tests and Holm’s correction.

For the instances from family **A**, the IVNS variant **baseline-first-t1.00** achieves lower gaps than VNS in the final evaluation, although VNS requires less time for the larger instances. For **F-n135-k7**, VNS outperforms IVNS in both solution quality and execution time. These results show that IVNS performance depends on the configuration and the test instance. Therefore, creating neighborhood structures during execution does not, by itself, guarantee an advantage over VNS.

Índice general

Dedicatoria	i
Agradecimientos	ii
Resumen	iv
Abstract	v
Introducción	1
1. Preliminares	4
1.1. Problema de Enrutamiento de Vehículos (VRP)	4
1.1.1. Ruta	5
1.1.2. Solución factible	6
1.2. Metaheurísticas	7
1.3. Vecindades y criterios de vecindad en problemas de VRP	8
1.4. Búsqueda de Vecindad Variable (VNS)	9
1.5. Instancias de prueba <i>benchmark</i> y análisis estadístico	11
1.5.1. Prueba de Wilcoxon	12
1.5.2. Prueba de Mann–Whitney	13
1.5.3. Prueba de Kruskal–Wallis	13
1.5.4. Prueba de Friedman	14
1.5.5. Corrección de Holm	14
1.5.6. Medidas del tamaño del efecto	15
1.6. Gramáticas libres de contexto	16
2. Búsqueda de Vecindad Infinitamente Variable	18
2.1. Búsqueda de Vecindad Infinitamente Variable (IVNS)	18
2.2. Gramática para la generación de vecindades	19
2.3. Hiperparámetros de la metaheurística IVNS	21

3. Diseño experimental y evaluación de algoritmos	25
3.1. Objetivos experimentales	26
3.2. Plataforma experimental y parametrización de los algoritmos	28
3.2.1. Parámetros de ejecución	29
3.3. Métricas	31
3.4. Diseño de experimentos	33
3.5. Validación de los datos y análisis estadístico	35
3.5.1. Comparaciones entre dos alternativas	36
3.5.2. Comparaciones entre tres o más alternativas	36
4. Resultados y discusión experimental	38
4.1. Comparación inicial entre VNS e IVNS	39
4.2. Comparación de las variantes de IVNS	41
4.3. Exploración del tiempo en las instancias A	43
4.4. Evaluación final en las instancias A	46
4.4.1. Calidad de las soluciones	47
4.4.2. Tiempo de ejecución	48
4.4.3. Estabilidad y relevancia práctica	48
4.5. Exploración y confirmación en F-n135-k7	49
4.5.1. Selección exploratoria	49
4.6. Discusión de las preguntas experimentales	52
4.7. Limitaciones	53
Conclusiones	56
Recomendaciones	59

Introducción

La distribución de alimentos, medicamentos, mercancías y combustible exige organizar recorridos, asignar clientes a una flota y establecer el orden de las visitas. Cuando aumenta el número de destinos, la enumeración manual de las alternativas deja de resultar viable. En consecuencia, la planificación de rutas ocupa un lugar relevante en la logística, el transporte y la gestión de recursos[20, 21].

El Problema de Enrutamiento de Vehículos (VRP, por sus siglas en inglés) estudia la construcción de rutas para que una flota atienda un conjunto de clientes desde uno o más depósitos. Cada recorrido comienza y termina en el depósito correspondiente. Entre sus variantes se encuentra el Problema de Enrutamiento de Vehículos con Restricción de Capacidad (CVRP), donde cada transporte dispone de un límite de carga y la demanda de cada ruta no supera ese valor. El objetivo consiste en atender una vez a cada cliente mediante recorridos factibles y minimizar la distancia total [19, 20, 21].

El CVRP pertenece a la optimización combinatoria. Su resolución requiere distribuir los puntos de demanda entre los vehículos y ordenar las visitas dentro de cada ruta. El número de alternativas crece con el tamaño de la instancia, lo que eleva el costo de explorar completamente el espacio de soluciones. Los métodos exactos garantizan la optimalidad cuando completan el proceso, mientras que las heurísticas y metaheurísticas buscan respuestas factibles de alta calidad sin enumerar todas las posibilidades [20, 22].

Las metaheurísticas para resolver el CVRP recurren con frecuencia a procedimientos de búsqueda local. Estos métodos parten de una solución vigente y ejecutan movimientos que originan alternativas cercanas. En el enrutamiento, una transformación reubica clientes, intercambia posiciones, modifica el orden de visita o traslada segmentos entre rutas. La regla que determina cómo se construyen los vecinos recibe el nombre de criterio de vecindad [22, 23].

Aunque la búsqueda local permite mejorar progresivamente una solución, su resultado depende del criterio de vecindad utilizado. Cuando ninguno de los movimientos de ese criterio reduce el costo, el procedimiento alcanza un óptimo local respecto a dicho entorno. Sin embargo, esa misma solución puede no constituir un óptimo para otra estructura de vecindad.

A partir de esta idea surge la Búsqueda de Vecindad Variable (VNS, por sus siglas

en inglés), una metaheurística que cambia sistemáticamente entre varias estructuras de vecindad durante el proceso de optimización. De este modo, cuando un entorno deja de aportar mejoras, el algoritmo explora otro y amplía la región examinada del espacio de soluciones. Esta variación recorre lugares diferentes del espacio de soluciones y facilita la salida de mínimos locales [24, 25].

En la VNS tradicional, el diseñador establece las vecindades y su orden antes de iniciar la ejecución. Por esta razón, el rendimiento depende de la elección de los movimientos y de la secuencia utilizada para recorrerlos. La Búsqueda de Vecindad Infinitamente Variable (IVNS, por sus siglas en inglés) amplía este principio mediante la construcción dinámica de criterios a partir de una gramática libre de contexto. En vez de operar únicamente con una lista fija, el método deriva cadenas que codifican combinaciones de operaciones sobre rutas, clientes y subrutas [4, 26].

Este enfoque resulta pertinente para el CVRP porque las formas de generar soluciones vecinas pueden representarse mediante secuencias de operaciones elementales: seleccionar una ruta, escoger un cliente o una subruta, insertar elementos y ejecutar intercambios. La gramática combina estas instrucciones y construye criterios durante cada ejecución. Sin embargo, la ampliación del repertorio no garantiza soluciones superiores, pues algunas secuencias son largas, difíciles de aplicar o costosas de evaluar.

Las diferencias entre las cadenas generadas por IVNS hacen necesario estudiar las reglas que orientan su construcción, la estrategia de búsqueda local, el tiempo asignado y las operaciones que se utilizan. Para evaluar estos elementos, la investigación compara IVNS con VNS mediante casos de prueba de la biblioteca de Problemas de Enrutamiento de Vehículos con Restricción de Capacidad (CVRPLIB por sus siglas en inglés) [?, ?]. Las ejecuciones utilizan semillas y, en los experimentos pareados, ambos métodos parten de la misma solución inicial. De esta manera, los resultados reflejan el comportamiento de los algoritmos y no las diferencias en el punto de partida.

El estudio comienza con una comparación inicial, continúa con una etapa exploratoria para seleccionar las variantes de IVNS y concluye con una evaluación confirmatoria mediante semillas nuevas. El análisis considera la calidad de las soluciones, el tiempo de ejecución y la estabilidad entre corridas, apoyándose en estadísticas descriptivas, pruebas no paramétricas, la corrección de Holm y tamaños del efecto. Este procedimiento permite determinar cómo la generación de cadenas, la búsqueda local y el tiempo disponible influyen en el rendimiento de IVNS [1, 2, 13, 3].

A partir de este planteamiento, el objetivo general consiste en evaluar experimentalmente el desempeño de IVNS aplicada al CVRP, mediante su comparación con una VNS tradicional y el análisis de la influencia de sus configuraciones sobre la calidad de las soluciones, el tiempo de ejecución y la estabilidad de los resultados.

Los objetivos específicos son los siguientes:

1. El estudio de los fundamentos teóricos del CVRP, VNS e IVNS.
2. El análisis de los criterios de vecindad utilizados por la búsqueda local para el CVRP y de su representación mediante una gramática libre de contexto.
3. La adaptación de una plataforma experimental para ejecutar VNS e IVNS bajo condiciones controladas y reproducibles.
4. El establecimiento de un diseño experimental con etapas inicial, exploratoria y confirmatoria que separe la selección de ajustes de su valoración posterior.
5. El análisis de la influencia de los perfiles de generación, las estrategias locales y los presupuestos temporales sobre el rendimiento de IVNS.
6. El registro y la validación del costo inicial, el mejor valor alcanzado, la brecha respecto al mejor costo conocido, la duración, las iteraciones, las mejoras aceptadas, la factibilidad y el número de cadenas generadas.
7. La comparación de los algoritmos y sus variantes mediante estadísticas descriptivas, pruebas no paramétricas, correcciones por contrastes múltiples y tamaños del efecto.
8. La determinación de las condiciones en las que IVNS resulta competitiva respecto de VNS y de las limitaciones observadas en los casos evaluados.

El documento se organiza en cuatro capítulos. El primero presenta los fundamentos del CVRP, las metaheurísticas basadas en búsqueda local, VNS, las herramientas estadísticas y las gramáticas libres de contexto. El segundo describe IVNS, su relación con el método tradicional, el formalismo utilizado para construir criterios y los hiperparámetros que regulan la ejecución. El tercero expone el diseño experimental, las instancias, las métricas, las variantes, las etapas y los procedimientos estadísticos. El cuarto examina y discute los resultados. Finalmente, las conclusiones sintetizan los hallazgos, delimitan las limitaciones y proponen recomendaciones para investigaciones futuras.

Esta investigación empleó herramientas de inteligencia artificial generativa como apoyo técnico. Su uso se concentró principalmente en la revisión y depuración del código para el análisis estadístico, en menor medida en algunas tareas de programación en Common Lisp de los modelos y, de forma puntual, en la revisión de la redacción.

Capítulo 1

Preliminares

En este capítulo se presentan los fundamentos teóricos necesarios para el desarrollo de este trabajo. En particular, se abordan los conceptos relacionados con el Problema de Enrutamiento de Vehículos con Restricción de Capacidad, las metaheurísticas basadas en métodos locales, la Búsqueda de Vecindad Variable, así como las herramientas estadísticas de utilidad para la validación experimental. Finalmente, se introducen las gramáticas libres de contexto como mecanismos para la generación de infinitas estructuras de vecindad.

1.1. Problema de Enrutamiento de Vehículos (VRP)

El Problema de Enrutamiento de Vehículos (VRP) es un modelo clásico de optimización combinatoria que surge en aplicaciones logísticas y de transporte [19, 20].

Su formulación general consiste en determinar rutas óptimas para una flota encargada de atender a (n) clientes desde (m) depósitos. Cada unidad parte de uno de los centros definidos y regresa a otro, mientras satisface las demandas y el conjunto de restricciones [19, 20].

El VRP no constituye un único problema, sino una familia de formulaciones que incorporan diferentes restricciones según las características de la aplicación. Entre sus variantes se encuentran el VRP con ventanas de tiempo, donde cada cliente debe ser atendido dentro de un intervalo; el VRP con recogidas y entregas, que combina el transporte de productos desde y hacia diferentes puntos; el VRP con retornos o *backhauls*, que incluye clientes de entrega y de recogida; y el VRP con múltiples depósitos, en el cual los vehículos operan desde distintos centros logísticos [20, 21].

Entre las variantes de esta familia, en este trabajo se estudia el Problema de

Enrutamiento de Vehículos con Restricción de Capacidad (CVRP). En esta formulación, cada vehículo dispone de una capacidad máxima y la demanda total de los clientes que pertenecen a una ruta no puede superar ese límite. El CVRP constituye una de las variantes más utilizadas en la literatura especializada y en la comparación de metaheurísticas [20, 21].

Para el CVRP clásico con un único depósito, el modelo se representa mediante un grafo completo ponderado ($G=(V,E)$), donde ($V=0,1,\dots,n$) reúne los nodos. El elemento (0) identifica el origen, mientras que los índices del (1) al (n) corresponden a los puntos de demanda de los clientes. Por su parte, (E) contiene las aristas entre cada par, con valores asociados (c_{ij}) que pueden expresar distancia, tiempo o costo de viaje entre (i) y (j).

A partir de esta representación puede precisarse cuándo una alternativa resulta factible. En la modalidad clásica, cada cliente recibe exactamente una visita de uno de los vehículos. Además, los recorridos comienzan y terminan en el nodo de origen, de modo que la flota salga desde un punto común, atienda los destinos y regrese a dicho lugar. La suma de las demandas de una ruta tampoco puede exceder el límite de carga de la unidad correspondiente. Estas condiciones distinguen al CVRP de otras variantes del VRP y determinan la construcción y evaluación de los resultados [19, 20].

Después de establecer las condiciones de factibilidad, el modelo también debe precisar qué aspecto busca optimizar. Cada variante del VRP, como el CVRP, puede perseguir distintos objetivos según las necesidades de la aplicación. Entre los más frecuentes se encuentran la minimización de la distancia, el tiempo de transporte y el costo de operación. Esta última magnitud puede incluir el consumo de combustible, el mantenimiento de los vehículos, los peajes y otros gastos del recorridos [20].

En esta investigación se adopta como función objetivo la minimización de la distancia total que recorren los vehículos. Esta medida se utiliza en las instancias de referencia del CVRP y permite comparar los resultados con los valores presentes en la literatura.

Las condiciones de factibilidad y el objetivo de optimización que presenta esta sección muestran que una solución del CVRP no consiste en un solo trayecto, sino en un conjunto de rutas que funciona de manera conjunta. Estas rutas atienden a todos los clientes sin superar la capacidad de los vehículos. Cada ruta indica el orden de las visitas desde la salida del depósito hasta el regreso a este. Por esta razón, antes de explicar una solución completa, conviene definir el concepto de ruta.

1.1.1. Ruta

Una ruta se define como una secuencia ordenada de nodos que describe el recorrido de un vehículo:

$$r = (0, v_1, v_2, \dots, v_k, 0).$$

donde (v_i) , para $(i=1, \dots, k)$, identifica el orden al que se visitan a los clientes.

En la variante clásica del CVRP, el vehículo sale del depósito, atiende los destinos y retorna al punto de origen [19, 20].

Una ruta es factible cuando comienza en el depósito, visita una secuencia de clientes, regresa al depósito y mantiene la suma de sus demandas dentro de la capacidad del vehículo. Una solución completa reúne todas las rutas necesarias para atender a cada cliente exactamente una vez. Además, ninguna ruta puede superar la capacidad del vehículo, ningún cliente puede quedar fuera y ningún cliente puede aparecer en más de una ruta. Por tanto, el conjunto de rutas factibles forma una solución completa del CVRP [19, 20].

1.1.2. Solución factible

Una solución factible del CVRP reúne un conjunto de rutas que cumple todas las restricciones del problema. El siguiente ejemplo muestra este concepto.

Considérese una instancia con las características siguientes:

- El nodo (0) representa el depósito.
- Los nodos (1,2,3,4,5) representan cinco clientes.
- La flota dispone de dos vehículos.
- Cada vehículo tiene una capacidad máxima de (10) unidades.
- Las demandas de los clientes son:

$$q_1 = 2, \quad q_2 = 4, \quad q_3 = 3, \quad q_4 = 2, \quad q_5 = 5.$$

Una posible solución contiene las rutas siguientes:

$$R_1 = (0, 2, 5, 0),$$

$$R_2 = (0, 1, 3, 4, 0).$$

La ruta R_1 sale del depósito, visita a los clientes (2) y (5), y regresa al depósito. La demanda que transporta este vehículo es:

$$q_2 + q_5 = 4 + 5 = 9.$$

Como $9 \leq 10$, la ruta R_1 respeta la capacidad del vehículo.

La ruta R_2 sale del depósito, visita a los clientes (1), (3) y (4), y regresa al depósito. La demanda correspondiente es:

$$q_1 + q_3 + q_4 = 2 + 3 + 2 = 7.$$

Como $7 \leq 10$, la ruta R_2 también respeta la capacidad del vehículo.

Además, la solución cumple las condiciones siguientes:

- Todos los clientes del (1) al (5) reciben una visita.
- Cada cliente aparece una sola vez.
- Las dos rutas comienzan y terminan en el depósito.
- Ningún vehículo supera su capacidad máxima.

Por tanto, el conjunto formado por R_1 y R_2 constituye una solución completa y factible del CVRP [19, 20].

Cada solución factible posee un costo que depende de la función objetivo. En esta investigación, dicho costo corresponde a la distancia total de las rutas. La solución óptima es aquella que alcanza el menor costo entre todas las soluciones factibles.

Sin embargo, el VRP pertenece a la clase de problemas NP-hard. Cuando aumenta la cantidad de clientes, también crece con rapidez el número de rutas y agrupaciones posibles. Por esta razón, la enumeración y la evaluación de todas las soluciones factibles pueden exigir un costo computacional muy alto [20, 21].

Ante esta dificultad, se emplean algoritmos que recorren el espacio de soluciones y buscan alternativas de alta calidad sin revisar todas las posibilidades. Entre estos métodos se encuentran las heurísticas y las metaheurísticas. La sección siguiente presenta los fundamentos de las metaheurísticas que utiliza esta investigación.

1.2. Metaheurísticas

Las metaheurísticas son estrategias generales para encontrar soluciones de alta calidad en problemas de optimización cuyo tratamiento mediante métodos exactos puede requerir un costo computacional elevado [22].

Un conjunto factible (S) contiene todas las soluciones que cumplen las restricciones del problema. En el caso del VRP, su tamaño crece factorialmente con el número de clientes [?], lo que dificulta una revisión exhaustiva [22].

Para recorrer este conjunto sin explorarlo por completo, las metaheurísticas emplean vecindades. Sea S el conjunto de soluciones factibles y $s \in S$ una solución de S .

Su vecindad, denotada por $N(s)$, reúne las soluciones s' que se obtienen mediante modificaciones locales, como reubicar, insertar o intercambiar clientes.

Sea $\mathcal{M}$ el conjunto de posibles movimientos. Cada elemento $m \in \mathcal{M}$ representa una operación que transforma la solución s

$$N(s) = \{s' \in S \mid s' = m(s), ; m \in \mathcal{M}\}. \quad (1.1)$$

La búsqueda local examina estas soluciones con el propósito de reducir el costo. Sea k el número de la iteración, s_k la solución actual y $f : S \rightarrow \mathbb{R}$ la función objetivo, donde un valor menor representa un mejor resultado. En la estrategia de mejor vecino, el algoritmo selecciona la solución de menor costo dentro de $N(s_k)$:

$$s_{k+1} = \arg \min_{s' \in N(s_k)} f(s'). \quad (1.2)$$

En esta expresión, s_{k+1} representa la solución elegida para la siguiente iteración. El algoritmo realiza el cambio cuando $f(s_{k+1}) < f(s_k)$. Si ningún elemento de $N(s_k)$ mejora el costo actual, s_k constituye un mínimo local respecto a su vecindad.

Este procedimiento favorece la explotación porque busca mejoras cerca de la solución actual mediante un mismo tipo de movimiento. Para ampliar la búsqueda, la metaheurística cambia la regla con la que produce los vecinos. Por ejemplo, si las reubicaciones de clientes no reducen el costo, puede pasar a intercambios, inversiones de segmentos o traslados de subrutas.

En VNS, este cambio ocurre cuando el algoritmo avanza de una vecindad N_k a la siguiente N_{k+1} . Además, la fase de sacudida aplica un movimiento aleatorio dentro de la vecindad que el algoritmo elige y crea otro punto de partida para la búsqueda local. Por tanto, el método no selecciona una solución completa al azar, sino que modifica la solución actual para examinar otra zona del espacio de soluciones.

Los conceptos anteriores permiten introducir uno de los elementos centrales de las metaheurísticas aplicadas al enrutamiento: las vecindades y los criterios para recorrer el dominio de búsqueda.

1.3. Vecindades y criterios de vecindad en problemas de VRP

En optimización combinatoria, una vecindad de una solución reúne las alternativas cercanas obtenidas mediante transformaciones locales que se aplican sobre la configuración de partida [22, 25].

En el contexto del VRP y sus variantes, estas alteraciones actúan sobre la estructura

de las rutas, cambiando la asignación de clientes o el orden en que el vehículo los visita. Cada resultado de estas operaciones constituye un vecino.

Para producir vecinos de una solución se tienen estas transformaciones elementales:

- Reubicación entre rutas: trasladar un cliente desde un recorrido hacia otro.
- Intercambio entre rutas: permutar dos clientes pertenecientes a recorridos diferentes.
- Cambio en el orden de visita: alterar la secuencia en que se atienden los nodos de un recorrido.
- Inserción en una nueva posición: mover un cliente hacia otro lugar del mismo recorrido o incorporarlo en uno diferente.
- Inversión parcial de segmentos: invertir el orden de una subsecuencia consecutiva dentro de una ruta.

La importancia de estas transformaciones radica en que determinan cómo se recorre el espacio de soluciones, que pueden producir avances mediante evaluaciones de bajo costo. En cambio, una composición más compleja facilita la salida de mínimos locales respecto a movimientos básicos, aun cuando existan alternativas superiores en otra región del dominio de búsqueda.

Los criterios de vecindad para CVRP pueden describirse como combinaciones de operaciones básicas sobre rutas y clientes. Por tanto, cada criterio no constituye una acción única, sino una secuencia de instrucciones aplicada sobre una solución.

Un ejemplo de una secuencia de instrucciones son la reubicación de clientes, el intercambio entre rutas y la inversión de segmentos los cuales pueden expresarse mediante estas instrucciones. La reubicación selecciona un cliente y lo inserta en otra posición; el intercambio identifica dos clientes y permuta sus lugares; y la inversión escoge una parte de la ruta y modifica su orden.

El uso coordinado de diferentes estructuras permite combinar movimientos elementales y compuestos durante el proceso de optimización. Entre los enfoques que emplea este principio destaca la Búsqueda de Vecindad Variable (VNS), cuyo funcionamiento general se presenta en la siguiente sección.

1.4. Búsqueda de Vecindad Variable (VNS)

La Búsqueda de Vecindad Variable (VNS) es una metaheurística que tiene como sustento el principio de que una solución óptima local para una estructura puede no conservar esa condición frente a otra [25].

En lugar de utilizar un solo criterio de vecindad durante toda la ejecución, VNS cambia entre varios criterios con el propósito de escapar de mínimos locales y continuar la reducción de la función objetivo [24, 25].

Dado un conjunto de vecindades:

$$N_1(s), N_2(s), \dots, N_k(s),$$

el algoritmo VNS sigue un esquema iterativo:

1. Generar una solución inicial s .
2. Inicializar $k = 1$.
3. Mientras $k \leq k_{\text{máx}}$:
 - Sacudida *shaking*: generar una alternativa $s' \in N_k(s)$ mediante la estructura N_k .
 - Aplicar búsqueda local sobre s' , para obtener el resultado mejorado s'' .
 - Si $f(s'') < f(s)$, actualizar $s = s''$ y reiniciar $k = 1$.
 - En caso contrario, aumentar $k = k + 1$ para examinar la estructura siguiente.

En términos generales, VNS parte de una solución inicial y examina las soluciones vecinas que produce cada criterio de vecindad. Si ninguna de ellas reduce el valor de la función objetivo, el algoritmo cambia al criterio siguiente. Después, la fase de sacudida escoge al azar una solución dentro de esa nueva vecindad y la usa como punto de partida para la búsqueda local.

La búsqueda local realiza la explotación, ya que busca mejoras cerca de la solución actual. Mientras que el cambio de criterio y la sacudida realizan la exploración, permitiendo que el algoritmo visite otras zonas del espacio de soluciones y evite el estancamiento en un mínimo local [24, 25].

El procedimiento local puede seguir dos estrategias:

- **Primera mejora**: acepta la primera alternativa cuyo valor supera al de la configuración vigente.
- **Mejor vecino**: examina todos los elementos del entorno y selecciona el que presenta el menor valor de la función objetivo.

El criterio de aceptación determina si el nuevo resultado sustituye al vigente. Para tomar esta decisión, se compara la calidad de ambos y se conserva aquel desde el cual continuará el algoritmo.

La combinación de la sacudida, el cambio entre criterios de vecindad y la búsqueda local caracteriza a VNS. A partir de esta metaheurística surgen extensiones que crean nuevos criterios durante la ejecución, entre ellas la Búsqueda de Vecindad Infinitamente Variable [24, 25, 4].

Después de explicar los fundamentos del CVRP y el funcionamiento general de VNS, conviene introducir las herramientas que permiten evaluar metaheurísticas en la resolución de problemas computacionales. Para ello, la sección siguiente presenta las instancias de prueba y los métodos estadísticos que permiten comparar la calidad, el tiempo y la estabilidad de distintas alternativas.

1.5. Instancias de prueba *benchmark* y análisis estadístico

Las instancias de prueba, *benchmark* por su nombre en inglés, corresponden a problemas estándar empleados para evaluar algoritmos. Su uso facilita el contraste entre técnicas sobre casos idénticos y mediante criterios comunes [11].

En el contexto del CVRP, estos referentes permiten examinar la calidad de las soluciones, el tiempo de cómputo y la estabilidad de cada alternativa.

Las metaheurísticas que incorporan decisiones aleatorias durante su ejecución, como la construcción de la solución inicial o la elección de movimientos, producen valores distintos entre corridas. Por esta razón, el estudio repite cada configuración con semillas diferentes y resume los datos mediante indicadores estadísticos.

Entre las medidas descriptivas más comunes en la literatura se encuentran la media, la mediana y la varianza [12]. La primera resume el comportamiento central del algoritmo sobre las repeticiones experimentales. La segunda representa una posición menos sensible a observaciones extremas, mientras que la tercera cuantifica la dispersión alrededor del valor medio.

Los intervalos de confianza estiman el rango que probablemente contiene el rendimiento medio real [12]. Esta información complementa la medida central, pues también interesa conocer la variabilidad entre corridas y, a partir de ella, valorar la estabilidad del procedimiento.

Las pruebas de hipótesis determinan si las diferencias en los resultados de los algoritmos, por ejemplo en calidad o tiempo de ejecución, reflejan comportamientos distintos o responden únicamente a la variación propia de los experimentos [12, ?]. En este contexto, una diferencia estadística aporta evidencia para respaldar esa conclusión [12].

Las herramientas estadísticas que utiliza este trabajo incluyen la prueba de Wilcoxon

[?], la prueba de Mann–Whitney [?], la prueba de Kruskal–Wallis [?] y la prueba de Friedman [?]. El análisis también aplica la corrección de Holm [?] y emplea medidas del tamaño del efecto, como el delta de Cliff [18] y la correlación biserial de rangos [15]. Las subsecciones siguientes explican sus fundamentos.

1.5.1. Prueba de Wilcoxon

La prueba Wilcoxon es no paramétrica y sirve comparar dos grupos de resultados relacionados [?]. Resulta adecuado cuando cada observación del primer conjunto mantiene una correspondencia directa con otra del segundo. Este procedimiento examina si las discrepancias favorecen de manera consistente a una alternativa sin exigir una distribución normal.

El análisis parte de pares (x_i, y_i) , donde x_i representa el resultado del primer algoritmo y y_i corresponde al segundo bajo el mismo caso experimental. La diferencia toma la forma:

$$d_i = x_i - y_i.$$

El signo de d_i muestra cuál alternativa obtuvo el mejor valor, mientras que su magnitud expresa la separación entre ambas. El procedimiento descarta los casos nulos, ordena $|d_i|$ de menor a mayor y asigna un rango r_i según esa posición.

Después de ordenar las diferencias absolutas y asignarles un rango r_i , el procedimiento suma por separado los rangos correspondientes a diferencias positivas y negativas:

$$W^+ = \sum_{d_i > 0} r_i, \quad W^- = \sum_{d_i < 0} r_i.$$

Estas sumas muestran si las discrepancias se concentran a favor de uno de los algoritmos. En un contraste bilateral, el estadístico corresponde al menor término:

$$W = \min(W^+, W^-).$$

Cuando una alternativa obtiene valores superiores, los rangos se acumulan en uno de los signos y el análisis identifica una diferencia estadística [?, ?].

La prueba de Wilcoxon requiere que cada observación de una muestra mantenga una relación directa con una observación de la otra muestra [?]. Cuando no existe esa correspondencia y las muestras son independientes, el análisis utiliza la prueba (U) de Mann–Whitney [?].

1.5.2. Prueba de Mann–Whitney

La prueba Mann–Whitney (U) evalúa dos muestras independientes [?]. Su aplicación corresponde a situaciones donde los datos del primer conjunto no guardan una relación directa con los elementos del segundo.

El procedimiento combina ambas muestras, ordena sus observaciones de menor a mayor y les asigna rangos. Si los tamaños respectivos son n_1 y n_2 , mientras R_1 expresa la suma de las posiciones correspondientes al primer conjunto, el estadístico toma la forma:

$$U_1 = n_1 n_2 + \frac{n_1(n_1 + 1)}{2} - R_1.$$

La expresión correspondiente a la segunda muestra es:

$$U_2 = n_1 n_2 - U_1.$$

En un análisis bilateral se toma el menor término:

$$U = \min(U_1, U_2).$$

Una separación sistemática entre los rangos de ambas muestras aporta evidencia de diferencias entre sus distribuciones [?].

La prueba de Mann–Whitney compara dos muestras independientes. Cuando el estudio examina al menos tres conjuntos independientes, Mann–Whitney deja de ofrecer un análisis global y se recurre a Kruskal–Wallis [?, ?].

1.5.3. Prueba de Kruskal–Wallis

La prueba Kruskal–Wallis examina al menos tres muestras independientes [?]. Su propósito consiste en determinar si las alternativas consideradas presentan diferencias globales sin requerir una relación entre sus observaciones.

Este procedimiento combina todos los datos, los ordena y asigna una posición a cada elemento. Si el análisis incluye k grupos, n_j expresa el tamaño de la muestra j , R_j representa la suma de sus rangos y N corresponde al total de registros, el estadístico toma la forma:

$$H = \frac{12}{N(N+1)} \sum_{j=1}^k \frac{R_j^2}{n_j} - 3(N+1).$$

La hipótesis nula sostiene que los grupos presentan la misma distribución. Si el estadístico (H) alcanza un valor suficientemente alto, la prueba rechaza esa hipótesis y concluye que al menos uno de los grupos difiere de los demás [?].

Kruskal–Wallis identifica la existencia de una discrepancia global, pero no determina los pares concretos que la originan. Para localizarlos se requieren contrastes posteriores. Cuando el análisis compara tres o más grupos relacionados, la prueba de Friedman permite realizar un contraste global entre ellos [?, ?].

1.5.4. Prueba de Friedman

La prueba Friedman analiza al menos tres muestras relacionadas [?, ?]. Su aplicación exige que los algoritmos o variantes actúen sobre los mismos casos experimentales.

Para cada unidad, el procedimiento ordena las alternativas según su resultado y asigna un rango. Si se evalúan k métodos sobre N casos, r_{ij} identifica la posición del algoritmo j en la unidad i . La clasificación media de cada opción queda definida por:

$$R_j = \frac{1}{N} \sum_{i=1}^N r_{ij}.$$

Friedman examina si estas posiciones medias presentan una separación suficiente para descartar un comportamiento equivalente. Su estadístico corresponde a:

$$\chi_F^2 = \frac{12N}{k(k+1)} \left[\sum_{j=1}^k R_j^2 - \frac{k(k+1)^2}{4} \right].$$

Bajo la hipótesis nula, las alternativas muestran clasificaciones semejantes. Si el estadístico supera el umbral correspondiente, el análisis rechaza dicha igualdad y reconoce diferencias entre los métodos [?, ?].

El contraste global solo indica que existe alguna diferencia entre los grupos, pero no señala qué pares la originan [?, ?]. Para identificar esos pares, el análisis aplica comparaciones posteriores. Como estas comparaciones aumentan el riesgo de detectar diferencias por azar, la corrección de Holm ajusta los niveles de significación y controla el error que surge al realizar varios contrastes [?, ?].

1.5.5. Corrección de Holm

Cuando un contraste global detecta diferencias, el análisis debe localizar las alternativas responsables. Para ello se realizan comparaciones por pares. Sin embargo, al efectuar (m) contrastes aumenta la probabilidad de declarar por azar una discrepancia inexistente, conocido como error por comparaciones múltiples [?, ?].

La corrección de Holm ajusta los niveles de significación y reduce ese riesgo [?]. El procedimiento ordena los valores (p) de menor a mayor:

$$p_{(1)} \leq p_{(2)} \leq \cdots \leq p_{(m)},$$

donde (m) representa el total de hipótesis examinadas. Después confronta cada $p_{(i)}$ con el umbral:

$$\frac{\alpha}{m - i + 1},$$

donde α corresponde al nivel de significación. Cuando una comparación no cumple la condición correspondiente, el procedimiento detiene la secuencia y no rechaza las hipótesis que siguen. De esta forma, la corrección de Holm reduce el riesgo de declarar diferencias por azar cuando el análisis incluye varios contrastes [?].

Las herramientas anteriores contrastan el comportamiento de las metaheurísticas y respaldan la interpretación experimental. En particular, determinan si las separaciones mantienen una dirección consistente en cada casos.

La significación estadística establece la existencia de evidencia, pero no cuantifica la magnitud del efecto. Por esta razón, el estudio incorpora medidas complementarias.

1.5.6. Medidas del tamaño del efecto

Las medidas del tamaño del efecto expresan la dirección y la magnitud de las diferencias entre algoritmos [18, 15]. Estas medidas complementan los valores (p), porque la significación estadística indica si los datos aportan evidencia contra la hipótesis nula, pero no informa por sí sola sobre la importancia práctica de la diferencia [?, 18].

Para datos relacionados, el análisis emplea la correlación biserial de rangos. A partir de las sumas positiva y negativa de Wilcoxon, esta medida toma la forma:

$$r_{rb} = \frac{W^+ - W^-}{W^+ + W^-}.$$

La correlación biserial de rangos toma valores entre (-1) y (1). Los valores cercanos a cero indican una diferencia pequeña, mientras que los valores próximos a los extremos indican una separación mayor entre los algoritmos. El signo señala cuál alternativa recibe la ventaja y depende del orden que el análisis asigna a las muestras [15].

Para comparar muestras independientes, el estudio utiliza el delta de Cliff. Esta medida compara cada observación del primer grupo con cada observación del segundo y calcula la diferencia entre la proporción de comparaciones favorables a un grupo y la proporción favorable al otro [18].

$$\delta = \frac{(x_i > y_j) - (x_i < y_j)}{n_x n_y}$$

El indicador también adopta cifras entre (-1) y (1). Una magnitud cercana a cero refleja comportamientos semejantes, mientras que la proximidad a cualquiera de los extremos señala un efecto más intenso. La dirección vuelve a depender del orden que

tengan las muestras.

Los contrastes y medidas respaldan la evaluación de las metaheurísticas. En conjunto, permiten elegir el procedimiento adecuado según la relación entre los datos, comprobar la consistencia de los efectos y estimar su magnitud.

Después de explicar los fundamentos de la evaluación experimental, conviene presentar el mecanismo con el que IVNS genera criterios de vecindad. IVNS utiliza una gramática libre de contexto para construir estas secuencias durante la búsqueda [4]. La sección siguiente introduce los fundamentos de este tipo de gramática [26].

1.6. Gramáticas libres de contexto

Antes de presentar el concepto de gramática, resulta necesario introducir las nociones de alfabeto, cadena y lenguaje.

Un alfabeto es un conjunto finito de símbolos disponibles para formar expresiones. Por ejemplo, $\Sigma = a, b, c$ contiene tres elementos.

Una cadena corresponde a una secuencia finita de componentes del alfabeto. Así, *abac* constituye una expresión formada sobre Σ .

Un lenguaje reúne las cadenas a partir del mismo conjunto inicial [26].

Una gramática establece las reglas necesarias para formar expresiones pertenecientes a una colección formal. Cada clase de lenguaje posee modelos gramaticales asociados [26]. En este trabajo se emplea una gramática libre de contexto, porque organiza secuencias de operaciones que producen criterios de vecindad dentro de la Búsqueda de Vecindad Infinitamente Variable (IVNS, por sus siglas en inglés). Esta extensión de VNS construye dinámicamente nuevas transformaciones durante el proceso de optimización [?].

Los compiladores, el procesamiento del lenguaje y los sistemas formales emplean estas estructuras para describir expresiones mediante símbolos e instrucciones precisas [26].

Formalmente, una gramática libre de contexto corresponde a la tupla:

$$G = (N, T, P, S),$$

donde (N) agrupa los símbolos no terminales, (T) reúne los terminales, $N \cap T = \emptyset$, $S \in N$ identifica el elemento inicial y (P) contiene las producciones.

Los terminales aparecen en las cadenas finales del lenguaje. En cambio, los no terminales actúan como componentes auxiliares que admiten nuevas transformaciones.

Cada producción adopta la forma:

$$A \rightarrow \alpha,$$

donde $A \in N$ y α es una secuencia compuesta por elementos de $N \cup T$.

La construcción comienza desde (S) y continúa mediante la aplicación sucesiva de las producciones disponibles.

Dada una gramática $(G=(N,T,P,S))$, con $\alpha, \beta \in (N \cup T)^*$, la relación $\alpha \Rightarrow \beta$ expresa que α deriva en β mediante un paso cuando existen $\gamma, \delta \in (N \cup T)^*$, un no terminal $A \in N$ y una producción $A \rightarrow \eta \in P$, tales que:

$$\alpha = \gamma A \delta \quad \text{y} \quad \beta = \gamma \eta \delta.$$

La notación $\Rightarrow^*$ indica una derivación compuesta por cero o más pasos. Por tanto, el lenguaje producido por (G) reúne las cadenas terminales obtenidas desde el símbolo inicial:

$$L(G) = \{ w \in T^* : S \Rightarrow^* w \}.$$

Así, (G) determina todas las expresiones que se forman de las producciones [26].

En el contexto de este trabajo, una secuencia puede describir operaciones aplicadas sobre una solución del CVRP. Por ejemplo, puede indicar la elección de una ruta, la identificación de un cliente y su reinserción en otra posición.

En lugar de enumerar manualmente un repertorio fijo de transformaciones, este mecanismo construye criterios de vecindad a partir de combinaciones válidas de operaciones. Esta propiedad constituye la base de la Búsqueda de Vecindad Infinitamente Variable, que se verá en el capítulo siguiente.

Los conceptos expuestos establecen la base necesaria para continuar la investigación. El CVRP determina el dominio de optimización; las metaheurísticas locales permiten explorar su espacio de soluciones; y el modelo gramatical aporta una representación formal para describir y construir secuencias de acciones. El próximo capítulo integra estos elementos mediante la presentación de IVNS.

Capítulo 2

Búsqueda de Vecindad Infinitamente Variable

Este capítulo analiza la Búsqueda de Vecindad Infinitamente Variable (IVNS) como una extensión de VNS [24, 25, 4]. El primer apartado explica la relación entre ambas metaheurísticas; el segundo presenta la gramática libre de contexto que IVNS usa para crear criterios de vecindad [4, 26]; y el tercero describe los hiperparámetros que controlan su ejecución [4, 22].

2.1. Búsqueda de Vecindad Infinitamente Variable (IVNS)

La Búsqueda de Vecindad Infinitamente Variable (IVNS, por sus siglas en inglés) surge como una extensión de la Búsqueda de Vecindad Variable (VNS) [?]. La propuesta conserva las fases de perturbación y búsqueda local, pero sustituye el repertorio finito y predefinido de estructuras por un procedimiento gramatical. Durante la ejecución, este formalismo combina operaciones elementales sobre rutas y clientes para construir nuevos criterios de transformación.

El enfoque reduce la dependencia de movimientos y amplía las alternativas disponibles en el proceso de optimización. La gramática incorpora dinámicamente reglas capaces de modificar la configuración vigente, lo que favorece el examen de otras regiones del espacio de soluciones [?].

La combinación de acciones elementales ofrece tres ventajas:

- Aumenta la diversidad de movimientos, porque el algoritmo no queda limitado a un repertorio definido previamente.

- Favorece la salida de mínimos locales al producir transformaciones distintas cuando las estructuras utilizadas dejan de mejorar el resultado.
- Incrementa la posibilidad de obtener soluciones de mayor calidad mediante el examen de regiones adicionales del dominio de búsqueda.

En términos generales, IVNS mantiene la lógica básica de VNS e incorpora la construcción dinámica de criterios mediante una gramática libre de contexto. El siguiente esquema resume el procedimiento:

1. Generar una solución inicial (s).
2. Mientras no se cumpla el criterio de parada:
 - Derivar una cadena válida mediante la gramática.
 - Interpretar la secuencia como un criterio de vecindad.
 - Aplicar la transformación sobre s para obtener una alternativa s' .
 - Ejecutar búsqueda local desde s' y producir s'' .
 - Si $f(s'') < f(s)$, actualizar $s = s''$.
 - En caso contrario, continuar la exploración con un nuevo criterio.

El elemento central del método radica en la construcción de cadenas y en su traducción a operaciones de vecindad. Por ello, antes de presentar los parámetros de IVNS, resulta necesario explicar la gramática que origina dichas secuencias.

2.2. Gramática para la generación de vecindades

Una de las principales aportaciones de IVNS consiste en emplear una gramática libre de contexto para construir criterios de vecindad [?]. Como se explicó en el capítulo anterior, las transformaciones en problemas de enrutamiento pueden expresarse mediante secuencias de operaciones elementales sobre rutas, clientes y subrutas.

Formalmente, la gramática de IVNS corresponde a la tupla:

$$G_{IVNS} = (N, T, P, S),$$

donde (N) agrupa los símbolos no terminales, (T) reúne los terminales, (P) contiene las producciones y (S) identifica el elemento inicial.

El repertorio de no terminales es:

$$N = S, S1, R1, R, A, B, C, F, G, E,$$

mientras que los terminales son:

$$T = r, a, b, c, f, g, e.$$

Cada elemento de (T) identifica una operación elemental que forma parte de un criterio de vecindad. En particular, (r) corresponde a la elección de una ruta; (a), a la identificación de un cliente; (b), a su inserción; y (c), al intercambio de dos clientes. Los símbolos (f), (g) y (e) actúan sobre subrutas: el primero escoge un segmento, el segundo lo inserta y el tercero intercambia dos de ellos.

El conjunto (P) establece las combinaciones válidas de estas acciones. En la implementación, sus reglas principales son:

$$S \rightarrow; RAS1B \mid RAS1RB \mid RAAS1C \mid RARAS1C \quad \mid RFS1G \mid RFS1RG \mid RFFS1E \mid RFRFS1E,$$

$$S1 \rightarrow; R1AS1R1B \mid R1AR1AS1C \mid R1FS1RG \quad \mid R1FR1FS1E \mid \varepsilon,$$

$$R1 \rightarrow R \mid \varepsilon,$$

y las producciones que convierten los no terminales en terminales son:

$$R \rightarrow r, \quad A \rightarrow a, \quad B \rightarrow b, \quad C \rightarrow c, \quad F \rightarrow f, \quad G \rightarrow g, \quad E \rightarrow e.$$

La derivación comienza en (S). El procedimiento elige una de sus reglas y sustituye sucesivamente los no terminales hasta obtener una expresión compuesta solo por elementos de (T). El símbolo (S1) extiende la secuencia mediante combinaciones adicionales, mientras que $S1 \rightarrow \varepsilon$ finaliza el proceso. De manera análoga, $R1 \rightarrow R \mid \varepsilon$ admite u omite una nueva elección de ruta en las posiciones correspondientes.

Por ejemplo, la derivación:

$$S \Rightarrow RAAS1C \Rightarrow RAAC \Rightarrow raac$$

produce (raac), criterio que elige una ruta, identifica dos clientes y ejecuta un intercambio. De forma similar:

$$S \Rightarrow RAS1B \Rightarrow RAB \Rightarrow rab$$

origina (rab), transformación que escoge un recorrido, identifica un cliente y lo inserta en una posición válida.

Cada expresión terminal derivada desde (S) corresponde a una combinación correcta

de operaciones elementales. Después, el algoritmo interpreta la secuencia y la aplica sobre una solución del CVRP para producir alternativas vecinas.

Luego de explicar la estructura gramatical y el procedimiento que construye los criterios, la siguiente sección analiza los parámetros que regulan el comportamiento de IVNS.

2.3. Hiperparámetros de la metaheurística IVNS

Los hiperparámetros influyen en el rendimiento de una metaheurística [22]. El usuario fija sus valores antes de iniciar la ejecución y, mediante ellos, controla la duración de IVNS, la generación de cadenas, la exploración de vecindades, la búsqueda local y la sacudida.

■ Límites de la ejecución completa

Estos tres hiperparámetros controlan toda la ejecución de IVNS:

- **Número máximo de iteraciones:** IVNS termina cuando el contador alcanza $I_{\text{máx}}$. Este valor limita la cantidad total de ciclos de la metaheurística.
- **Límite total de tiempo:** el usuario fija el tiempo máximo para toda la ejecución. IVNS termina cuando el reloj alcanza ese valor.
- **Umbral de ciclos sin mejora:** IVNS cuenta los ciclos consecutivos en los que no reduce la función objetivo. El proceso termina cuando el contador alcanza el umbral.

■ Límite de tiempo por vecindad

Este hiperparámetro no controla toda la ejecución. Actúa cuando IVNS toma una cadena de la gramática, la convierte en un criterio y aplica ese criterio sobre la solución actual s .

Durante ese intervalo, IVNS crea soluciones vecinas, comprueba su factibilidad y calcula sus costos. Cuando el reloj alcanza el límite, el algoritmo interrumpe la exploración de ese criterio y continúa con el resto de la metaheurística.

Por tanto, los tres hiperparámetros anteriores restringen IVNS en su conjunto, mientras que el límite de tiempo por vecindad restringe solo la creación y la evaluación de vecinos para un criterio concreto.

■ Generación y control de las cadenas

La gramática combina acciones sobre rutas, clientes y subrutas para crear criterios de vecindad. Los parámetros siguientes controlan este proceso:

- **Número máximo de intentos:** el generador puede realizar una cantidad máxima de intentos para crear una cadena válida. Si no lo consigue, abandona ese proceso y IVNS continúa con la acción siguiente.
- **Longitud máxima:** una cadena con ℓ símbolos terminales representa ℓ operaciones elementales. Cuando ℓ aumenta, también pueden aumentar la complejidad del criterio y el costo de la exploración.
- **Comprobación de las cadenas:** antes de usar una cadena, el sistema comprueba que pertenezca al lenguaje de la gramática, que cumpla las restricciones estructurales y que el intérprete pueda convertirla en una secuencia coherente de operaciones sobre una solución del CVRP. Si alguna comprobación falla, el sistema descarta la cadena y realiza otro intento.

Estos parámetros actúan durante la creación de la cadena. El límite de tiempo por vecindad comienza después, cuando IVNS ya posee un criterio y empieza a crear y evaluar vecinos sobre s .

■ Control de la búsqueda local

La búsqueda local utiliza dos límites:

- **Número máximo de rondas:** este parámetro limita la cantidad de rondas que puede ejecutar la búsqueda local.
- **Umbral de rondas sin mejora:** la búsqueda local cuenta las rondas consecutivas que no reducen el costo. Cuando el contador alcanza el umbral, el procedimiento local termina.

En cada ronda, la búsqueda local examina soluciones vecinas y busca una alternativa con menor costo que la solución actual.

■ Modalidad de búsqueda local

IVNS puede recorrer los vecinos mediante tres modalidades:

- **Mejor vecino (best):** examina todos los candidatos y conserva el que alcanza el menor costo.
- **Primera mejora (first):** acepta el primer candidato que reduce el costo de la solución actual.
- **Selección aleatoria (random):** cambia al azar el orden de los movimientos antes de examinarlos.

■ Historial de cadenas

Un hiperparámetro determina qué ocurre con el historial después de cada mejora:

- Si el usuario activa el reinicio, IVNS borra el historial y permite que las cadenas anteriores vuelvan a aparecer.
- Si el usuario conserva el historial, IVNS excluye las cadenas que ya utilizó.

Esta decisión modifica la variedad de criterios que IVNS puede explorar después de una mejora. Ningún parámetro fija directamente el número total de cadenas. Esa cantidad depende del número de iteraciones, los límites de tiempo, el máximo de intentos, la longitud máxima y las comprobaciones de la gramática.

■ Categorías de sacudida

Los parámetros de perturbación controlan el alcance de la sacudida (*shaking*). IVNS contempla dos categorías:

- **Sacudida débil:** aplica cambios pequeños, como mover un cliente, cambiar su posición dentro de una ruta o insertarlo en otra. Estas acciones conservan gran parte de la solución actual y permiten explorar soluciones cercanas.
- **Sacudida fuerte:** aplica cambios de mayor alcance, como trasladar una subruta, modificar dos rutas o cambiar un segmento completo. Estas acciones alteran una parte mayor de la solución y permiten explorar zonas más lejanas del espacio de soluciones.

La sacudida débil concentra la búsqueda cerca de la solución actual. La sacudida fuerte permite que IVNS visite otras regiones cuando los cambios pequeños no generan mejoras. ““

En conjunto, estos hiperparámetros regulan las distintas fases de IVNS y determinan cómo la metaheurística distribuye el esfuerzo entre la generación de criterios, la exploración de vecinos, la búsqueda local y el cambio hacia otras regiones del espacio de soluciones.

Además el estudio evalúa tres modalidades de recorrido:

- **Mejor vecino best:** examina todos los candidatos disponibles y conserva el de menor costo.
- **Primera mejora first:** acepta el primer candidato cuyo valor mejora la configuración vigente.

- **Selección aleatoria random:** ordena aleatoriamente los movimientos antes de examinarlos.

El ajuste de estos hiperparámetros equilibra exploración y explotación, dos componentes que influyen en el rendimiento de IVNS.

En resumen, este capítulo precisó las características que distinguen a IVNS dentro de los métodos basados en vecindades. La propuesta conserva la combinación de perturbación y búsqueda local propia de VNS, pero incorpora una gramática libre de contexto para construir criterios dinámicos. El capítulo también explicó la traducción de las cadenas a operaciones sobre soluciones del CVRP y describió los parámetros que regulan la ejecución. Estos contenidos aportan la base teórica para presentar, en el capítulo siguiente, el diseño experimental y el análisis de los resultados.

Capítulo 3

Diseño experimental y evaluación de algoritmos

Este capítulo describe el diseño experimental para contrastar el desempeño de las metaheurísticas VNS e IVNS en la resolución del Problema de Enrutamiento de Vehículos con Restricción de Capacidad. El análisis considera la calidad de las soluciones, la duración de las corridas, su estabilidad y, para IVNS, el efecto del presupuesto temporal, los tres perfiles de generación y las estrategias de búsqueda local.

La experimentación comprende tres etapas:

- **Etap 1. Comparación inicial entre VNS e IVNS:** esta etapa establece una referencia del comportamiento de ambos métodos antes del análisis de los parámetros de IVNS. La comparación examina la calidad de las soluciones y el tiempo de ejecución.
- **Etap 2. Exploración de variantes de IVNS:** esta etapa compara combinaciones de ajustes de generación, estrategias de búsqueda local y límites de tiempo por vecindad. El análisis busca la variante que ofrece la relación más favorable entre la calidad de las soluciones y el costo computacional.
- **Etap 3. Evaluación final frente a VNS:** esta etapa compara con VNS la variante que alcanzó el mejor balance durante la exploración. Las ejecuciones usan semillas que no formaron parte de la etapa 2, con el propósito de comprobar si el comportamiento de la variante se mantiene bajo otras secuencias pseudoaleatorias.

El apartado Objetivos experimentales establece las dimensiones del estudio y las preguntas que lo orientan. Luego, Plataforma experimental y parametrización de los

algoritmos detalla el entorno de ejecución, los componentes del sistema y los parámetros de cada corrida. Métricas reúne las medidas utilizadas para examinar la calidad, el tiempo de ejecución y la estabilidad. Por su parte, Diseño de experimentos explica la organización de la comparación inicial, la exploración de variantes y los bloques confirmatorios. Finalmente, Validación de los datos y análisis estadístico expone las comprobaciones de los registros y los procedimientos para contrastarlos. Esta última parte distingue el análisis de dos métodos del correspondiente a tres o más grupos, según la cantidad de opciones y la relación entre las observaciones.

La organización anterior requiere definir primero los elementos que orientan el estudio. La subsección siguiente presenta sus objetivos experimentales.

3.1. Objetivos experimentales

Esta subsección precisa las dimensiones de las pruebas. El propósito es contrastar VNS e IVNS sobre el CVRP mediante la calidad de las soluciones, el tiempo de ejecución y la estabilidad entre corridas. Para ello, se registran el costo final, la brecha porcentual respecto al mejor valor conocido, la duración computacional y la dispersión de los datos. Estas métricas permiten evaluar algoritmos y metaheurísticas que incorporan decisiones aleatorias durante la búsqueda [1, 2, 3].

La primera etapa proporciona una referencia inicial entre VNS e IVNS. La fase exploratoria examina nueve variantes de IVNS y estudia el presupuesto temporal, los perfiles de generación y las estrategias de búsqueda local con el fin de identificar una combinación candidata. La confirmación utiliza corridas adicionales y semillas diferentes para comprobar el comportamiento de la opción elegida. De este modo, la selección permanece separada de la valoración posterior [1, 2].

El estudio de IVNS incluye tres perfiles de generación. El perfil base (**baseline**) utiliza la distribución predeterminada del generador y prioriza las operaciones elementales. El perfil de complejidad media (**efg-medium**) asigna a los símbolos (e), (f) y (g), asociados a operaciones sobre subrutinas, un peso superior al establecido en **baseline**. El perfil de complejidad alta (**efg-high**) incrementa esa preferencia respecto a **efg-medium**. Los tres perfiles comparten la misma gramática y solo difieren en los parámetros que orientan la construcción de las secuencias [4].

De la gramática se producen cadenas que representa un criterio de vecindad. El algoritmo no fija directamente el total de expresiones a construir o evaluar. Esta cantidad depende del presupuesto temporal, del máximo de intentos y de los filtros que descartan estructuras inválidas o incompatibles con la solución vigente [4].

Los casos de prueba que utiliza esta investigación proceden de la biblioteca CVRPLIB [5]. El estudio toma casos de dos familias:

- **Familia A:** la investigación utiliza A-n33-k5, A-n65-k9 y A-n80-k10. Augerat y sus colaboradores definieron esta familia de casos para el estudio del CVRP [6].
- **Familia F:** la investigación utiliza F-n135-k7. Fisher propuso este caso, que contiene 135 nodos y requiere siete rutas en su solución de referencia [7].

La nomenclatura de CVRPLIB resume las características principales de cada caso. La letra inicial identifica la familia, (n) indica la cantidad total de nodos y (k) expresa el número de rutas o vehículos de la solución de referencia. La cantidad de nodos también cuenta el depósito. Por ejemplo, A-n33-k5 contiene 33 nodos y cinco rutas, mientras que F-n135-k7 contiene 135 nodos y siete rutas.

Con los objetivos del estudio, los ajustes de generación y este conjunto de casos, la investigación plantea las siguientes preguntas:

1. ¿Existen diferencias entre VNS e IVNS en la calidad de las soluciones sobre las instancias?
2. ¿Qué diferencias presentan VNS e IVNS en el tiempo de ejecución sobre las instancias?
3. ¿Las diferencias aparecen de forma consistente entre las corridas o dependen de casos aislados?
4. ¿Qué diferencias producen las estrategias de búsqueda local mejor vecino (**best**), primera mejora (**first**) y selección aleatoria (**random**) en la calidad de las soluciones y el tiempo de ejecución de IVNS?
5. ¿Las variantes de generación que favorecen cadenas más complejas mejoran el desempeño de IVNS?
6. ¿Aumentar el tiempo disponible para explorar una vecindad permite obtener soluciones de mayor calidad?
7. ¿Una variante seleccionada durante la fase exploratoria mantiene su comportamiento al evaluarse con semillas nuevas?
8. ¿La exploración de F-n135-k7 selecciona la misma variante de IVNS que la exploración realizada sobre las instancias A?

Una vez definidas las preguntas del estudio, la siguiente subsección detalla el sistema y los parámetros de las ejecuciones.

3.2. Plataforma experimental y parametrización de los algoritmos

El estudio utiliza una plataforma que ejecuta las corridas de VNS e IVNS bajo las condiciones para cada bloque. El sistema organiza las ejecuciones, permite modificar los parámetros, asigna semillas pseudoaleatorias y registra la información para analizar los resultados.

La implementación de VNS e IVNS utiliza Common Lisp [8]. SBCL funciona como compilador y entorno de ejecución [9]. Python permite limpiar y validar los datos, aplicar los procedimientos estadísticos y generar las tablas y figuras [10]. El repositorio de GitHub contiene el código de la plataforma y los archivos para reproducir los experimentos.¹

La arquitectura experimental comprende cuatro componentes: carga de las instancias, construcción de las soluciones iniciales, ejecución de los algoritmos y registro de los resultados.

Carga de las instancias del problema. Este componente lee los datos de cada caso del CVRP: clientes, demandas, capacidad de los vehículos, coordenadas de los nodos y mejor valor conocido. El sistema trabaja con A-n33-k5, A-n65-k9, A-n80-k10 y F-n135-k7.

Construcción de las soluciones iniciales. Este componente produce el punto de partida de la búsqueda. Una solución se considera factible cuando cada cliente aparece exactamente una vez, todas las rutas comienzan y terminan en el depósito y ninguna supera la capacidad del vehículo.

En la etapa 3, el sistema construye una solución inicial para cada combinación de caso y semilla. Luego crea dos copias iguales y entrega una a VNS y otra a IVNS. Cada solución inicial presenta un identificador único. Para conformarlo, el sistema utiliza la distribución de los clientes entre las rutas, el orden de visita, el costo y el procedimiento de construcción. Este identificador permite comprobar que ambos algoritmos parten del mismo estado.

Ejecución de los algoritmos. Este componente ejecuta VNS e IVNS. Para estudiar la segunda metaheurística, el experimento combina tres ajustes de generación, **baseline**, **efg-medium** y **efg-high**, con tres estrategias de búsqueda local, **best**, **first** y **random**. Esta combinación produce nueve variantes de IVNS:

¹<https://github.com/Livi005/C-digo-Tesis>

3 ajustes de generación $\times$ 3 estrategias de búsqueda local = 9 variantes.

VNS recorre un conjunto de vecindades definido antes de comenzar la búsqueda. En cambio, IVNS utiliza una gramática para construir durante la ejecución cadenas que representan criterios de vecindad [4].

La plataforma también separa las corridas exploratorias de las confirmatorias. Las primeras permiten seleccionar una variante de IVNS, mientras que las segundas utilizan semillas nuevas para comprobar si el comportamiento se mantiene.

Registro de los resultados. El sistema genera un archivo de resumen y otro de traza. El resumen contiene una fila por corrida y constituye la fuente principal del análisis presentado en el capítulo de resultados. La traza conserva la evolución de cada ejecución y funciona como registro auxiliar.

El archivo de resumen incluye la instancia, el algoritmo, la semilla, el perfil de generación, la estrategia local, el presupuesto temporal por vecindad, el costo inicial, el mejor costo, la brecha porcentual, el tiempo de ejecución, la factibilidad y el estado final de la corrida.

3.2.1. Parámetros de ejecución

Cada corrida se controla mediante parámetros antes de comenzar el experimento. Algunos regulan tanto VNS como IVNS, mientras que otros responden a las características propias de cada método. Esta distinción permite identificar qué condiciones comparten los algoritmos y cuáles se modifican durante el estudio de IVNS.

Parámetros comunes a VNS e IVNS. El número máximo de iteraciones fija la cantidad límite de repeticiones del ciclo principal. En VNS, cada iteración comprende una sacudida, una fase de búsqueda local y la decisión de aceptar o rechazar el resultado. IVNS conserva esta estructura, pero obtiene el criterio de vecindad mediante una cadena de la gramática. En ambos casos, este parámetro controla los ciclos externos y no representa el total de vecinos.

El límite de iteraciones sin mejora establece cuántos ciclos pueden transcurrir sin reducir el costo. Cuando el contador alcanza el valor definido, el algoritmo detiene la corrida para evitar que la búsqueda continúe sin avances.

Ambos métodos también utilizan una estrategia de búsqueda local. La modalidad de mejor vecino (**best**) examina los candidatos y conserva el de menor costo. La primera mejora (**first**) acepta el primer movimiento que reduce la función objetivo. Por su

parte, la selección aleatoria (**random**) modifica de forma pseudoaleatoria el orden en que se examinan los movimientos.

El número máximo de rondas locales limita los ciclos internos de la búsqueda local antes de regresar al ciclo principal. A su vez, el límite de rondas sin mejora detiene esta fase cuando transcurren varios ciclos consecutivos sin encontrar una solución de menor costo.

Cuando el experimento incluye un límite total de ejecución, este parámetro establece la duración máxima de la corrida. Su función no debe confundirse con el tiempo asignado a cada vecindad en IVNS.

La semilla pseudoaleatoria controla las decisiones que dependen del azar y permite reproducir una ejecución. Sin embargo, utilizar la misma semilla no demuestra por sí solo que VNS e IVNS comiencen desde una solución idéntica, porque ambos métodos pueden consumir los valores pseudoaleatorios de forma diferente.

El método de construcción de la solución inicial determina el procedimiento para obtener el punto de partida. En las pruebas finales, el sistema crea una solución para cada combinación de instancia y semilla, produce dos copias y entrega una a cada algoritmo. El identificador de la solución inicial, calculado a partir de la distribución y el orden de los clientes en las rutas, junto con su costo y el procedimiento de construcción, permite comprobar que VNS e IVNS parten del mismo estado.

Parámetro exclusivo de VNS. El nivel máximo de vecindad, representado por $k_{\text{máx}}$, determina cuántas estructuras predefinidas puede recorrer VNS. Cuando el método encuentra una solución de menor costo, reinicia el recorrido desde la primera vecindad. Este parámetro no tiene el mismo significado en IVNS, porque la segunda metaheurística no recorre una lista fija de estructuras, sino que construye criterios durante la ejecución.

Parámetros exclusivos de IVNS. IVNS utiliza varios controles relacionados con la generación de cadenas. La gramática produce secuencias pseudoaleatorias que representan operaciones aplicables sobre la solución vigente. El máximo de intentos limita la cantidad de pruebas hasta obtener una cadena que cumpla las condiciones establecidas.

Los ajustes de generación **baseline**, **efg-medium** y **efg-high** modifican la frecuencia con la que aparecen determinadas operaciones. **baseline** utiliza una distribución; **efg-medium** incrementa la presencia de los símbolos asociados a operaciones sobre subrutas; y **efg-high** aumenta aún más esa preferencia. Los tres ajustes emplean la misma gramática y solo cambian los valores que orientan la construcción de las cadenas [4].

Los filtros descartan las expresiones que no pueden convertirse en criterios ejecutables. Estos controles restringen la longitud de las cadenas, comprueban la cantidad y el

orden de los símbolos, mantienen el equilibrio entre las acciones y excluyen estructuras incompatibles con la solución actual.

IVNS también conserva un historial temporal de las cadenas utilizadas para reducir las repeticiones durante la búsqueda. Después de encontrar una solución de menor costo, el parámetro de reinicio determina si el algoritmo elimina ese historial y permite utilizar nuevamente las cadenas desde el nuevo punto de partida.

El tiempo por vecindad establece la duración máxima que IVNS puede dedicar a construir y evaluar los vecinos asociados a un criterio. Este control resulta importante porque las cadenas con más operaciones pueden requerir una evaluación más costosa. La fase exploratoria modifica este valor para examinar su efecto sobre la calidad de las soluciones y el tiempo de ejecución.

Ningún parámetro fija directamente la cantidad total de cadenas generadas o evaluadas. Esta cifra depende del máximo de intentos, los filtros, el tiempo de cada vecindad y el costo de aplicar cada criterio.

Una vez explicados los parámetros comunes y los controles particulares de cada método, la siguiente subsección presenta las métricas para valorar la calidad de las soluciones, el tiempo de ejecución y la variabilidad entre corridas.

3.3. Métricas

El estudio registra indicadores asociados a la calidad de las soluciones, la duración de las corridas, la estabilidad de los datos y la dinámica interna de IVNS. Estas medidas examinan el valor que cada método alcanzó, la evolución de la búsqueda y la variabilidad entre ejecuciones [11, 12].

Costo inicial. Corresponde al valor de la función objetivo de la solución que inicia la búsqueda. Este indicador identifica el punto de partida de cada corrida y comprueba, en los experimentos, que VNS e IVNS reciben configuraciones iniciales equivalentes.

Mejor costo. Identifica el menor valor de la función objetivo en el algoritmo en una corrida. Esta medida refleja la calidad de la mejor solución y constituye uno de los criterios principales para contrastar VNS e IVNS.

Media de los mejores costos. Resume el promedio de los valores mínimos en las repeticiones de una misma configuración. El indicador describe el rendimiento general del método dentro del bloque experimental correspondiente.

Mediana del costo. Expresa el valor central de los mejores costos obtenidos. Esta medida reduce la influencia de observaciones extremas y facilita el contraste entre

algoritmos y variantes.

Desviación estándar. Cuantifica la dispersión de los datos alrededor de la media. Una magnitud inferior refleja mayor estabilidad entre corridas.

Cuartiles y rango intercuartílico. Describen la región central de la distribución y su dispersión sin depender directamente de los extremos.

Tiempo de ejecución. Mide la duración de una corrida y permite analizar el costo computacional de cada procedimiento. En IVNS, el presupuesto disponible también condiciona el número de cadenas y vecinos que el método genera y evalúa.

Brecha porcentual. Cuantifica la separación relativa entre el mejor costo obtenido y el mejor valor conocido de la instancia. La siguiente expresión define esta medida:

$$\text{gap}(\%) = \frac{c_{\text{obt}} - \text{BKS}}{\text{BKS}} \times 100,$$

donde c_{obt} corresponde al costo alcanzado por el algoritmo y mejor costo conocido en la literatura BKS identifica el mejor costo conocido. Esta normalización facilita la comparación entre instancias con escalas diferentes.

Número de iteraciones. Indica cuántos ciclos principales completa el algoritmo antes de que alguna condición de parada termine la ejecución. Este valor muestra la extensión del proceso para cada ajuste.

Número de mejoras aceptadas. Registra cuántas veces una alternativa de menor costo sustituye a la configuración vigente. Este conteo permite estudiar el progreso del procedimiento.

Factibilidad de la solución. Comprueba el cumplimiento de las restricciones del CVRP. El análisis final incorpora únicamente corridas completas con resultados factibles.

Diferencias entre pares. En los experimentos confirmatorios, este indicador compara los valores de VNS e IVNS asociados a cada punto de partida compartido. El registro también incluye las victorias, derrotas y empates de ambos métodos.

Métricas de convergencia. Describen la evolución del costo por iteración, el mejor valor acumulado, los avances aceptados y los períodos de estancamiento.

Métricas internas de IVNS. Caracterizan el funcionamiento del generador gramatical. Incluyen la longitud y complejidad medias, además del total de cadenas

válidas, inválidas, evaluadas, seleccionadas, aceptadas y únicas. También contabilizan vecinos no factibles, ausencia de mejora y agotamiento del presupuesto temporal.

Una vez definidos los indicadores, la siguiente subsección explica la organización de los experimentos y su aplicación en cada contraste.

3.4. Diseño de experimentos

Esta subsección explica cómo se organiza el estudio para responder las preguntas experimentales. En cada bloque, el diseño especifica el algoritmo, el caso de prueba, el ajuste de generación, la estrategia de búsqueda local y, cuando resulta necesario, el límite de tiempo por vecindad.

En este estudio, una corrida o ejecución corresponde a la aplicación completa de un algoritmo sobre una instancia con una semilla y unos parámetros. En las evaluaciones finales, un par reúne una corrida de VNS y otra de IVNS que parten de copias de una misma solución inicial.

El límite de tiempo por vecindad indica la cantidad máxima de segundos que IVNS puede dedicar a generar y evaluar los vecinos de un criterio. Este valor no representa la duración total de la corrida, sino el tiempo para examinar cada vecindad construida por la gramática.

La experimentación comprende tres fases generales: comparación inicial, exploración y evaluación final. La primera proporciona una referencia del rendimiento de VNS e IVNS antes de seleccionar un ajuste específico. La segunda estudia las variantes de IVNS y permite elegir una candidata. La tercera utiliza semillas nuevas para comprobar el comportamiento de la combinación seleccionada.

La Tabla 3.1 resume los bloques del estudio y la cantidad de ejecuciones de cada uno.

En la etapa 1, VNS e IVNS resuelven los casos **A-n33-k5**, **A-n65-k9** y **A-n80-k10**, que pertenecen a la familia **A** de CVRPLIB [6]. IVNS utiliza la variante que combina el ajuste base con la estrategia de selección aleatoria (**baseline-random**). Cada algoritmo realiza 30 ejecuciones por caso con el fin de examinar la variabilidad que introduce el componente pseudoaleatorio [1, 2]. Esta etapa comprende 180 ejecuciones: 90 corresponden a VNS y 90 a IVNS.

El estudio de las variantes de IVNS combina tres ajustes de generación: el ajuste base (**baseline**), el ajuste de complejidad media (**efg-medium**) y el ajuste de complejidad alta (**efg-high**). Estos se combinan con tres estrategias de búsqueda local: mejor vecino (**best**), primera mejora (**first**) y selección aleatoria (**random**). Las combinaciones entre

Cuadro 3.1: Resumen de los experimentos

Experimento	Organización	Total
Comparación inicial en A	Dos algoritmos, tres instancias y 30 ejecuciones por combinación	180
Evaluación de variantes de IVNS en A	Tres ajustes de generación, tres estrategias locales, tres instancias y 30 ejecuciones por combinación	810
Exploración del tiempo en A	Nueve variantes, cuatro límites de tiempo, tres instancias y 30 ejecuciones por combinación	3240
Evaluación final en A	Dos algoritmos, tres instancias y 30 ejecuciones por combinación	180
Prueba inicial en F-n135-k7	Dos ajustes de generación, tres estrategias locales y cinco ejecuciones por combinación	30
Exploración de complejidad alta en F-n135-k7	Tres estrategias locales, cinco límites de tiempo y cinco ejecuciones por combinación	75
Evaluación final en F-n135-k7	Dos algoritmos, una instancia y 30 ejecuciones por combinación	60
Total general		4575

los tres ajustes y las tres estrategias producen nueve variantes. Cada una de la 9 variantes realiza realiza 30 ejecuciones sobre las tres instancias A, lo que genera un total de 810.

La exploración del tiempo sobre las instancias **A** combina las nueve variantes con límites de tiempo por vecindad de (0,20), (0,50), (1,00) y (1,50) segundos. Cada combinación realiza 30 ejecuciones en cada una de las tres instancias. Este bloque comprende 3240 ejecuciones y permite examinar la calidad de las soluciones y el tiempo de ejecución.

La etapa 2 selecciona la variante que combina el ajuste base, la estrategia de primera mejora y un límite de (1,00) segundo por vecindad (**baseline-first-t1.00**).

La evaluación final sobre las instancias A utiliza semillas entre 31 y 60, diferentes a la etapa de la exploración. Para cada combinación de instancia y semilla, el sistema construye una solución inicial, crea dos copias idénticas y entrega una a VNS y otra a IVNS. El identificador, el costo y el procedimiento de construcción de la solución permiten comprobar la igualdad del punto de partida. Este bloque contiene 30 pares por instancia y 180 ejecuciones en total.

La instancia **F-n135-k7** recibe una exploración independiente. El piloto combina los ajustes **baseline** y **efg-medium** con las tres estrategias locales. Cada una de las seis combinaciones realiza cinco corridas, para un total de 30 ejecuciones.

Después, el estudio del ajuste **efg-high** combina las tres estrategias locales con

límites de (0,05), (0,25), (0,50), (1,00) y (1,50) segundos por vecindad. Cada combinación realiza cinco corridas, lo que produce 75 ejecuciones.

Esta exploración selecciona la variante `efg-high-first-t1.50`, formada por el ajuste de complejidad alta, la estrategia de primera mejora y un límite de (1,50) segundos por vecindad. La evaluación final posterior utiliza semillas nuevas, comparte una solución inicial dentro de cada par y comprende 60 ejecuciones.

El sistema conserva los datos en archivos de resumen y traza. El archivo de resumen contiene una fila por corrida con las métricas principales y constituye la fuente de datos para el análisis estadístico. La traza registra la evolución de las ejecuciones y funciona como información auxiliar para comprobar su desarrollo.

Cada experimento modifica únicamente los factores relacionados con su supuesto. Los demás parámetros permanecen constantes dentro de cada bloque. La siguiente sección presenta los procedimientos utilizados para validar y analizar los datos.

3.5. Validación de los datos y análisis estadístico

Esta subsección explica el procedimiento que permite comprobar la integridad de los registros y elegir las pruebas estadísticas para cada comparación. El análisis utiliza las métricas que presenta la Sección ??.

Antes de aplicar los contrastes, el proceso de limpieza y validación comprobó cinco condiciones: integridad de las corridas, factibilidad de las soluciones, cálculo correcto de la brecha porcentual, ausencia de registros duplicados y exclusión de ejecuciones de prueba.

En las comparaciones entre VNS e IVNS, la validación también verificó la correspondencia entre las observaciones. Un par resulta válido cuando ambas corridas pertenecen a la misma instancia y comienzan desde una solución inicial idéntica. La semilla, el identificador de la solución inicial, el costo y la procedencia confirman esa igualdad.

La cantidad de alternativas y la relación entre las observaciones determinan la prueba estadística a emplear. El estudio usa un nivel de significación $\alpha = 0,05$, mientras que el procedimiento de Holm ajusta los valores (p) derivados de comparaciones múltiples [12, ?, ?].

Al completar la validación y establecerse la relación entre las muestras, el análisis comienza con los contrastes entre dos alternativas, como los realizados entre VNS e IVNS.

3.5.1. Comparaciones entre dos alternativas

Cuando cada corrida de una alternativa mantuvo una correspondencia verificable con una observación de la otra, el estudio aplicó la prueba de Wilcoxon para muestras pareadas [?]. Esta condición se cumplió en los contrastes oficiales y confirmatorios entre VNS e IVNS, pues ambos métodos comenzaron desde la misma solución inicial.

Para cada instancia, el análisis examinó la brecha porcentual y el tiempo de ejecución. Además del valor (p), calculó la variación entre los integrantes de cada par, la mediana de esas separaciones, el número de victorias, derrotas y empates, así como la correlación biserial de rangos. Estos indicadores permiten valorar la existencia del efecto, su dirección, magnitud y consistencia [15].

La relación entre las observaciones se determina a partir del diseño del experimento y de los datos de cada corrida. Dos resultados se consideran relacionados cuando existe una correspondencia uno a uno entre una ejecución de VNS y otra de IVNS sobre la misma instancia, con la misma repetición y copias de una solución inicial idéntica. Esta igualdad se comprueba mediante el identificador de la solución inicial, su costo y el procedimiento utilizado para construirla.

Cuando no existe una correspondencia directa entre las observaciones, o las corridas parten de soluciones iniciales diferentes, los grupos se tratan como muestras independientes y se comparan mediante la prueba (U) de Mann–Whitney [?]. La coincidencia de las semillas por sí sola no demuestra que las muestras estén relacionadas, porque los algoritmos pueden utilizar los valores pseudoaleatorios de manera diferente [12, ?].

Ante tres o más alternativas, los procedimientos anteriores no proporcionan un contraste global. En esa situación, el análisis recurre a herramientas para evaluar conjuntamente todos los grupos.

3.5.2. Comparaciones entre tres o más alternativas

Para evaluar tres o más configuraciones independientes, el estudio aplicó la prueba de Kruskal–Wallis [?]. Este procedimiento examinó las variantes de generación de cadenas, las estrategias de búsqueda local y los límites temporales considerados en IVNS.

Cuando el contraste global detectó evidencia de comportamientos diferentes, el análisis posterior comparó cada par mediante Mann–Whitney (U). La corrección de Holm ajustó los valores (p) resultantes y controló el incremento del error asociado a la realización de múltiples contrastes [?, ?, ?].

La interpretación también consideró las medidas descriptivas y el delta de Cliff para muestras independientes. De esta manera, las conclusiones no dependen exclusivamente de la significación estadística, sino que incorporan la dirección, la magnitud y la relevancia práctica del efecto observado [18].

Este procedimiento define los criterios para validar los datos y contrastar los algoritmos. El capítulo siguiente analiza la calidad de las soluciones, el tiempo de ejecución, la estabilidad entre corridas y el comportamiento de las configuraciones de IVNS.

Capítulo 4

Resultados y discusión experimental

Este capítulo examina los resultados de VNS e IVNS sobre las instancias del CVRP. El análisis conserva la organización anterior y se estructura en tres fases generales: comparación inicial, exploración de variantes y evaluación final.

Los resultados se analizan desde dos perspectivas. La primera compara VNS con IVNS. La segunda examina solo IVNS y contrasta las variantes al combinar sus ajustes de generación, estrategias de búsqueda local y límites de tiempo por vecindad.

La comparación proporciona una referencia del comportamiento de ambos métodos. La exploración estudia distintas variantes de IVNS y permite seleccionar una de ellas según la calidad de las soluciones y el tiempo de ejecución. Por último, la evaluación final compara la variante de IVNS con VNS mediante corridas que utilizan semillas diferentes de las que utiliza la exploración.

La semilla no modifica los datos de la instancia, como la cantidad de nodos, las demandas, las distancias o la capacidad de los vehículos. Su función consiste en inicializar el generador pseudoaleatorio del sistema. Por ello, semillas distintas pueden producir soluciones iniciales diferentes y modificar decisiones durante la búsqueda, como los movimientos de sacudida, el orden de exploración de los vecinos y las cadenas de la gramática que emplea IVNS.

El uso de semillas nuevas en la evaluación final permite comprobar si el algoritmo obtuvo o no buenos resultados únicamente debido a las soluciones iniciales o a las decisiones aleatorias de las corridas durante su elección. Esta comprobación estudia la estabilidad del comportamiento frente a la variabilidad aleatoria, pero no permite generalizar los resultados a instancias distintas de las analizadas.

Antes de interpretar los datos, el proceso de validación comprobó la integridad de los registros, la factibilidad de las soluciones y la igualdad del punto de partida en las comparaciones relacionadas. En estas comparaciones, cada semilla genera una solución

inicial y el sistema entrega copias idénticas de ella a VNS e IVNS. De esta forma, ambos métodos parten del mismo estado dentro de cada par. El capítulo anterior describió este procedimiento.

El apartado Comparación inicial entre VNS e IVNS proporciona una referencia del rendimiento de ambos métodos antes de seleccionar una variante específica de la segunda metaheurística (una combinación concreta de ajuste de generación y estrategia de búsqueda local). Luego, Comparación de las variantes de IVNS estudia las nueve combinaciones que resultan al relacionar tres ajustes de generación con tres estrategias de búsqueda local.

Exploración del tiempo en las instancias A analiza cuatro límites de tiempo por vecindad e identifica la variante utilizada posteriormente. El apartado Evaluación final en las instancias A contrasta esa variante con VNS mediante semillas nuevas y soluciones iniciales compartidas dentro de cada par.

A continuación, Exploración y evaluación final en F-n135-k7 presenta la selección y comparación de una variante de IVNS sobre esta instancia de 135 nodos, perteneciente a la familia F. Debido a que el estudio incluye una sola instancia de esta familia, sus resultados se interpretan únicamente para F-n135-k7.

El apartado Discusión de las preguntas experimentales relaciona los hallazgos con los interrogantes en el capítulo anterior. Finalmente, Limitaciones delimita el alcance de las conclusiones y expone las condiciones que deben considerarse al interpretarlas.

La comparación inicial establece el punto de partida del análisis.

4.1. Comparación inicial entre VNS e IVNS

La comparación inicial estableció una referencia del rendimiento de VNS e IVNS antes de explorar otras variantes de la segunda metaheurística. Cada algoritmo realizó 30 ejecuciones sobre los casos de prueba A-n33-k5, A-n65-k9 y A-n80-k10.

En esta comparación, IVNS utilizó la variante `baseline-random`, que combina el ajuste de generación base con la estrategia local de selección aleatoria. Las 30 ejecuciones de cada caso conservaron los mismos hiperparámetros; únicamente cambió la semilla pseudoaleatoria. Para cada caso y semilla, el sistema creó una solución inicial y entregó una copia a VNS y otra a IVNS.

La configuración estableció 10 iteraciones, $k_{\text{máx}} = 2$, cinco ciclos sin mejora, cinco intentos de sacudida, tres rondas de búsqueda local, dos rondas locales sin mejora y un límite total de 60 segundos. Para IVNS, la configuración también estableció 50 intentos de generación, (0,05) segundos por vecindad y el reinicio del historial de cadenas tras cada mejora.

La calidad de las soluciones se examinó mediante la brecha porcentual con respecto

a la mejor solución de referencia. Esta medida indica cuánto se aleja el costo que alcanza cada algoritmo del mejor valor de referencia para el caso de prueba. Por tanto, una brecha menor representa una solución de mayor calidad.

La mediana resume el valor central de las 30 ejecuciones y reduce la influencia de resultados extremos. La Tabla 4.1 presenta las medianas de la brecha y los valores (p) que produce la prueba de Wilcoxon después de la corrección de Holm.

Cuadro 4.1: Resultados de calidad en la comparación inicial

Caso	IVNS	VNS	p_{Holm}	Resultado
A-n33-k5	67,17	54,08	0,000306	VNS presenta menor brecha
A-n65-k9	148,17	136,84	0,000037	VNS presenta menor brecha
A-n80-k10	135,59	124,99	0,002766	VNS presenta menor brecha

En los tres casos, VNS alcanzó una mediana de brecha menor que IVNS. Los valores p_{Holm} quedaron por debajo de (0,05), por lo que el análisis detectó diferencias significativas en las tres comparaciones.

El análisis del tiempo de ejecución no mostró la misma dirección en los tres casos. La Tabla 4.2 presenta las medianas del tiempo y los valores (p) que produce la prueba de Wilcoxon después de la corrección de Holm.

Cuadro 4.2: Resultados de tiempo en la comparación inicial

Caso	IVNS (s)	VNS (s)	p_{Holm}	Resultado
A-n33-k5	0,232	0,315	$9,31 \times 10^{-8}$	IVNS requiere menos tiempo
A-n65-k9	0,328	0,299	0,001131	VNS requiere menos tiempo
A-n80-k10	0,415	0,302	$5,59 \times 10^{-9}$	VNS requiere menos tiempo

En A-n33-k5, IVNS alcanzó la menor mediana de tiempo. En A-n65-k9 y A-n80-k10, VNS terminó antes. Los valores p_{Holm} quedaron por debajo de (0,05), por lo que el análisis detectó diferencias significativas en los tres casos.

Por consiguiente, la variante inicial **baseline-random** no superó a VNS en la calidad de las soluciones. En cuanto al tiempo, IVNS presentó una ventaja en A-n33-k5, mientras que VNS presentó una ventaja en los otros dos casos.

Estos resultados no permiten establecer una conclusión definitiva sobre IVNS. Solo indican que **baseline-random**, antes de la exploración de otras configuraciones, alcanzó una calidad inferior a la de VNS en esta comparación. También muestran que la creación dinámica de criterios de vecindad no garantiza por sí sola una mayor calidad en los casos de prueba que estudia esta etapa.

A partir de esta referencia, el apartado siguiente examina cómo cambian los resultados de IVNS al combinar distintos ajustes de generación y estrategias de búsqueda local.

4.2. Comparación de las variantes de IVNS

Este apartado compara las variantes de IVNS para examinar cómo cambian la calidad de las soluciones y el tiempo de ejecución al modificar los ajustes de generación y las estrategias de búsqueda local.

El estudio combinó tres ajustes de generación: base (**baseline**), complejidad media (**efg-medium**) y complejidad alta (**efg-high**); con tres estrategias locales: mejor vecino (**best**), primera mejora (**first**) y selección aleatoria (**random**). La combinación de estos factores produjo nueve variantes:

$$3 \text{ ajustes de generación} \times 3 \text{ estrategias locales} = 9 \text{ variantes.}$$

Cada variante realizó 30 ejecuciones sobre cada una de las tres instancias A. La comparación utilizó la brecha porcentual y el tiempo de ejecución. En ambas métricas, una cifra menor representa un mejor resultado.

Las pruebas de Kruskal–Wallis detectaron diferencias estadísticamente significativas entre las nueve variantes en la brecha porcentual y el tiempo de ejecución de las tres instancias, con ($p < 0,001$) en todos los casos. Este resultado indica que al menos una de las variantes presentó una distribución diferente, aunque no significa que todas las parejas difirieran entre sí.

La Figura 4.1 permite comparar las nueve variantes en los tres casos de prueba. Las variantes de **baseline** alcanzan las brechas menores, las variantes de **efg-medium** ocupan una posición intermedia y las variantes de **efg-high** concentran los valores mayores. La separación entre los ajustes de generación resulta más visible que las diferencias entre las estrategias **first**, **best** y **random**.

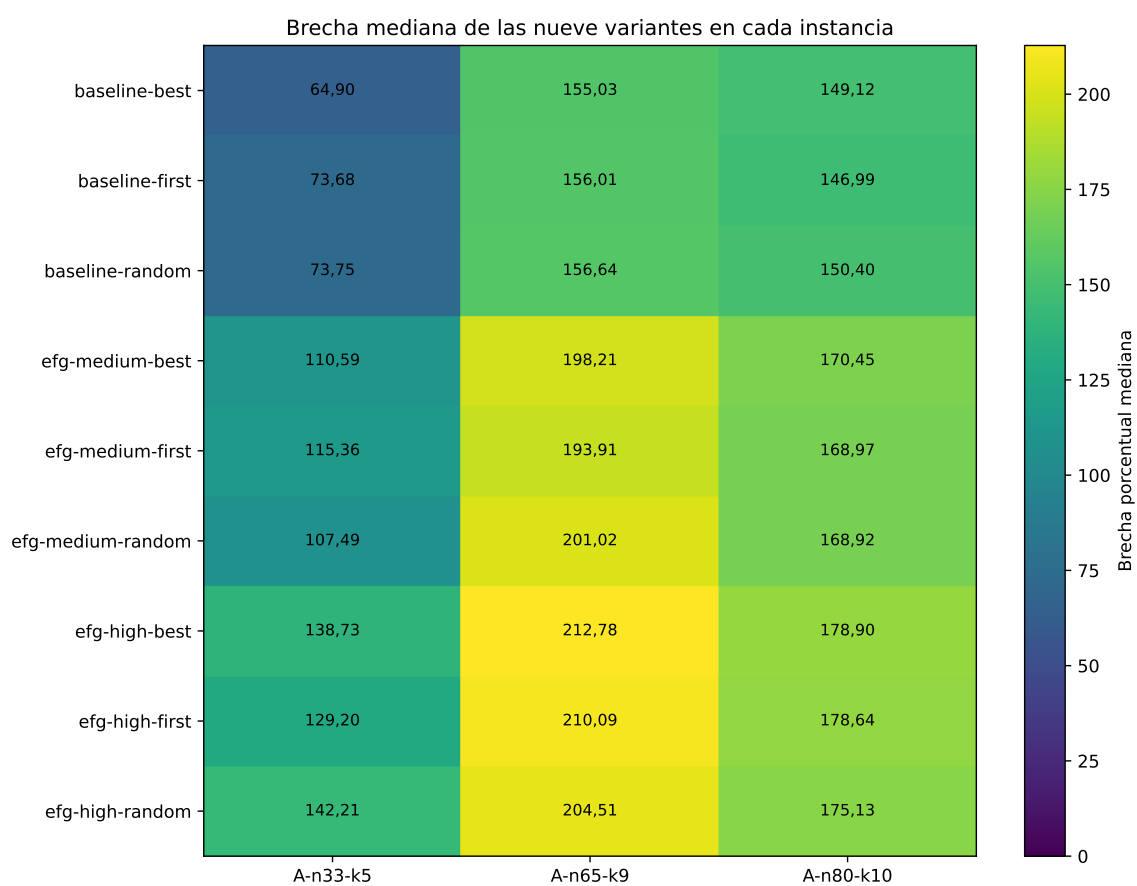


Figura 4.1: Brecha porcentual mediana de las nueve variantes de IVNS en los casos de la familia A.

La Tabla 4.3 resume este comportamiento mediante el menor y el mayor valor mediano alcanzado por las tres estrategias dentro de cada ajuste de generación.

Cuadro 4.3: Intervalos de las medianas de brecha porcentual según el ajuste de generación

Ajuste de generación	A-n33-k5	A-n65-k9	A-n80-k10
Base	64,90–73,75	155,03–156,64	146,99–150,40
Complejidad media	107,49–115,36	193,91–201,02	168,92–170,45
Complejidad alta	129,20–142,21	204,51–212,78	175,13–178,90

Por ejemplo, en **A-n33-k5**, las medianas de las variantes con el ajuste base se encontraron entre (64,90, %) y (73,75, %). Para el ajuste de complejidad media, los valores estuvieron entre (107,49, %) y (115,36, %); y para el ajuste de complejidad alta, entre (129,20, %) y (142,21, %). Las otras dos instancias conservaron el mismo orden general.

Los tiempos medianos conservaron el mismo orden general. Las variantes de **baseline** requirieron menos tiempo, las variantes de **efg-medium** ocuparon una posición intermedia y las variantes de **efg-high** alcanzaron los mayores tiempos en los tres casos.

Los resultados muestran que los ajustes que aumentan la presencia de los símbolos (e), (f) y (g), asociados a operaciones sobre subrutas, no mejoraron la calidad bajo las condiciones estudiadas. Al contrario, las variantes de complejidad media y alta presentaron brechas mayores y requirieron más tiempo que las variantes con el ajuste base.

Este resultado describe el comportamiento de las combinaciones evaluadas. No permite atribuir todas las diferencias exclusivamente al ajuste de generación o a la estrategia local, porque cada variante combina ambos elementos. Sin embargo, la repetición del mismo orden en las tres instancias muestra que el ajuste de generación fue el factor que produjo la separación más visible entre los grupos.

En la clasificación conjunta de las nueve variantes, las tres primeras posiciones correspondieron a **baseline-random**, **baseline-best** y **baseline-first**. Después se ubicaron las tres variantes de complejidad media y, finalmente, las tres de complejidad alta.

El siguiente apartado analiza si aumentar el límite de tiempo por vecindad mejora el rendimiento de las variantes de IVNS sobre las instancias A.

4.3. Exploración del tiempo en las instancias A

La fase exploratoria relacionó las nueve variantes de IVNS con cuatro límites de tiempo por vecindad: (0,20,s), (0,50,s), (1,00,s) y (1,50,s).

Las nueve variantes fueron **baseline-first**, **baseline-best**, **baseline-random**, **efg-medium-first**, **efg-medium-best**, **efg-medium-random**, **efg-high-first**, **efg-high-best** y **efg-high-random**.

Cada variante utilizó los cuatro límites en los casos A-n33-k5, A-n65-k9 y A-n80-k10. Cada combinación realizó 30 ejecuciones por caso.

El límite de tiempo por vecindad no establece la duración total de una ejecución. IVNS activa esta fase varias veces durante la búsqueda. En cada activación, el algoritmo dispone como máximo del límite correspondiente para generar y evaluar los vecinos de un criterio de vecindad.

La fase puede concluir antes cuando IVNS encuentra una mejora, agota las alternativas o cumple otra condición de parada. Por esta razón, el límite por vecindad no equivale a la duración total del algoritmo.

El análisis examinó si un límite mayor permitía generar y evaluar alternativas con una brecha menor. También buscó la combinación con la relación más favorable entre la calidad de las soluciones y el tiempo de ejecución.

En el análisis global, la variante **baseline-first-t1.00** ocupó el primer lugar. Esta variante combina el ajuste de generación **baseline**, la estrategia local **first** y un límite de (1,00,s) por vecindad.

En las 90 ejecuciones que corresponden a los tres casos, esta variante alcanzó una brecha media de (98,99), una brecha mediana de (105,90) y un tiempo mediano de (0,91,s). La Tabla 4.4 presenta la variante que alcanzó la menor brecha mediana dentro de cada límite.

Cuadro 4.4: Variante con la menor brecha mediana en cada límite de tiempo

Límite (s)	Variante	Brecha mediana
0,20	baseline-best	121,84
0,50	baseline-first	114,77
1,00	baseline-first	105,90
1,50	baseline-first	106,18

La tabla resume únicamente la variante que alcanzó la menor brecha dentro de cada límite. El experimento también examinó las otras ocho variantes en cada valor temporal.

La Figura 4.2 amplía la información de la tabla anterior, pues muestra las nueve variantes para cada límite de tiempo. Las variantes de **baseline** mantienen las brechas menores, mientras que **efg-medium** y **efg-high** alcanzan valores mayores. La disminución más clara aparece al aumentar el límite hasta (1,00,s); el cambio hasta (1,50,s) no genera una mejora general en la calidad.

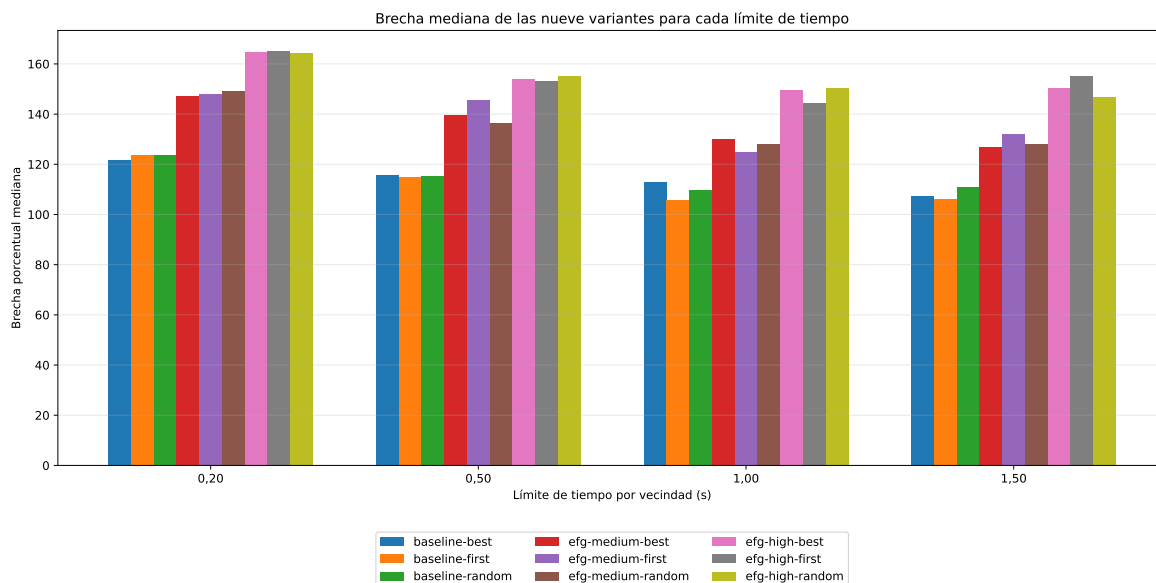


Figura 4.2: Brecha porcentual mediana de las nueve variantes de IVNS según el límite de tiempo por vecindad.

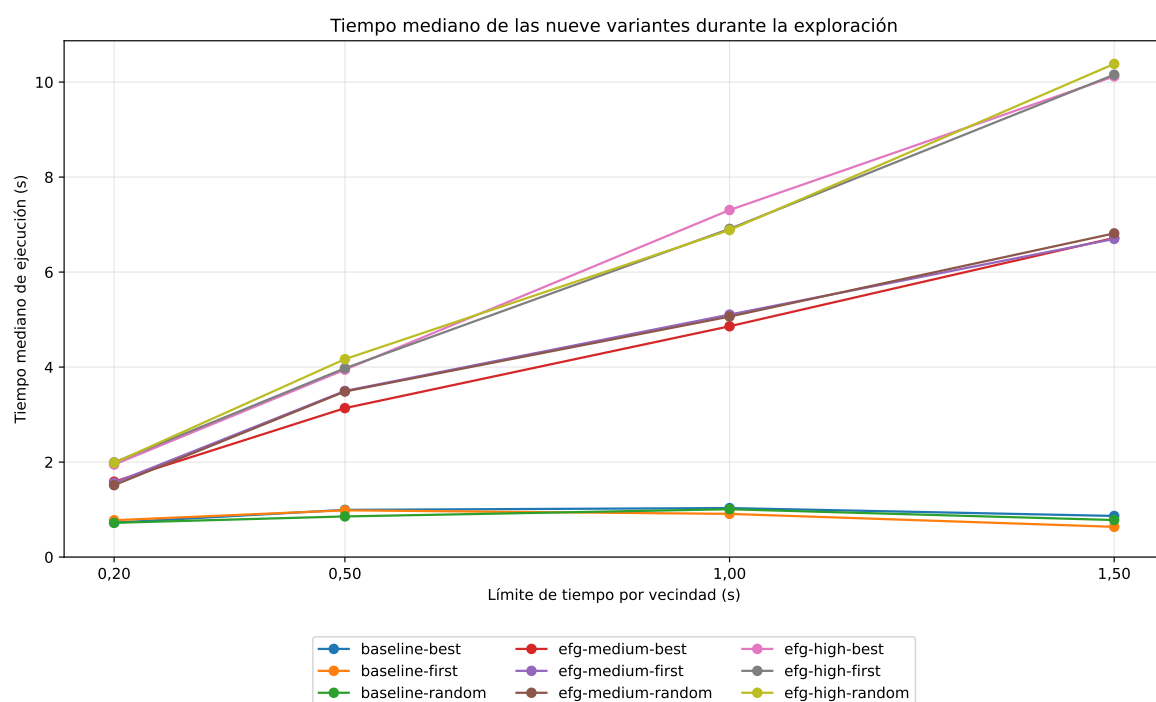


Figura 4.3: Tiempo mediano de ejecución de las nueve variantes de IVNS según el límite de tiempo por vecindad.

La Figura 4.3 muestra que el tiempo crece cuando aumenta el límite por vecindad. Este crecimiento resulta menor en las variantes de **baseline** y adquiere mayor intensidad en **efg-medium** y **efg-high**. Por tanto, las variantes que favorecen secuencias más complejas requieren más tiempo sin alcanzar brechas menores que las variantes del

ajuste base.

El efecto del límite dependió del caso de prueba. En **A-n33-k5**, el contraste no detectó cambios significativos en la calidad cuando aumentó el tiempo por vecindad.

En **A-n65-k9**, los límites de (1,00,s) y (1,50,s) alcanzaron brechas menores que el límite de (0,20,s). En **A-n80-k10**, esos dos límites alcanzaron brechas menores que los límites de (0,20,s) y (0,50,s).

El contraste entre (1,00,s) y (1,50,s) no detectó diferencias significativas en **A-n65-k9** ni en **A-n80-k10**. Por esta razón, el análisis eligió (1,00,s). Este límite alcanzó una calidad semejante a la que alcanzó (1,50,s), pero concedió medio segundo menos a cada exploración.

La selección también favoreció el ajuste **baseline**, que genera secuencias de menor complejidad. En los tres casos de la familia A, **baseline-first-t1.00** ofreció una relación más favorable entre la cercanía a la mejor solución de referencia y el tiempo de ejecución que las combinaciones que favorecen expresiones gramaticales más complejas.

Tras esta elección, el apartado siguiente compara **baseline-first-t1.00** con VNS mediante semillas que la fase exploratoria no utilizó.

4.4. Evaluación final en las instancias A

La evaluación final comparó la variante **baseline-first-t1.00** con VNS mediante semillas diferentes de las empleadas durante la exploración. En cada par de ejecuciones, ambos algoritmos recibieron copias de una misma solución inicial. De esta forma, las diferencias dentro de cada par no se deben a que uno de los métodos comenzara desde un punto de partida favorable.

Esta comparación no repite el experimento inicial. En aquella etapa se utilizó una variante inicial de IVNS, **baseline-random**, antes de explorar sus parámetros. En cambio, la evaluación final utiliza **baseline-first-t1.00**, seleccionada después de comparar distintas combinaciones de ajustes de generación, estrategias locales y límites de tiempo por vecindad.

Por tanto, el cambio en los resultados no representa una contradicción. La comparación inicial mostró que la variante de ese momento era inferior a VNS. La evaluación final examina otra variante de IVNS, la que mejor comportamiento mostró durante la exploración, y comprueba sus resultados mediante nuevas ejecuciones aleatorias. Debido a que también se emplearon semillas diferentes, la variación entre ambas etapas no debe atribuirse exclusivamente a un único parámetro, aunque resulta coherente con la mejora durante la selección de la variante.

4.4.1. Calidad de las soluciones

La calidad se evaluó mediante la brecha porcentual respecto a la mejor solución conocida. Una cifra menor indica una solución de mayor calidad. La Tabla 4.5 presenta las medianas obtenidas por cada método y el número de pares en los que IVNS alcanzó una brecha inferior.

Cuadro 4.5: Resultados finales de calidad en las instancias A

Instancia	IVNS	VNS	Victorias de IVNS	Resultado estadístico
A-n33-k5	57,03	66,79	19 de 30, con 1 empate	Significativo
A-n65-k9	130,20	158,69	30 de 30	Significativo
A-n80-k10	111,60	136,50	30 de 30	Significativo

Los valores de las columnas IVNS y VNS son las medianas de la brecha porcentual de las 30 ejecuciones. La columna de victorias indica cuántas veces IVNS obtuvo una brecha menor que VNS dentro de los pares contruidos a partir de una misma solución inicial.

Para interpretar las diferencias pareadas se calculó, en cada par, la resta:

$$\text{brecha de IVNS} - \text{brecha de VNS}.$$

Un resultado negativo favorece a IVNS, porque indica que su brecha fue menor. Estas diferencias se expresan en la diferencia directa entre los porcentos, ya que representan la separación entre dos valores porcentuales.

En A-n33-k5, la mediana de las 30 diferencias pareadas fue de (-8,93) puntos porcentuales. Esto significa que, en el centro de la distribución de los pares, la brecha de IVNS fue aproximadamente (8,93) puntos porcentuales menor que la de VNS. Después de aplicar la corrección de Holm, el valor ajustado fue ($p=0,002060$). La correlación biserial de rangos alcanzó (0,655) y favoreció a IVNS.

La mediana de las diferencias pareadas no tiene que coincidir exactamente con la resta de las medianas generales en la tabla. Primero se calcula la diferencia dentro de cada uno de los 30 pares y después se obtiene la mediana de esas diferencias.

En A-n65-k9, IVNS alcanzó una brecha inferior en los 30 pares. La mediana de las diferencias fue de (-26,45) puntos porcentuales y el tamaño del efecto alcanzó el máximo valor favorable a IVNS.

La misma tendencia apareció en A-n80-k10. IVNS obtuvo una brecha menor en las 30 comparaciones pareadas y la mediana de las diferencias fue de (-28,02) puntos porcentuales.

En consecuencia, la variante de IVNS obtuvo soluciones más cercanas a la mejor solución conocida que VNS en las tres instancias A. Las pruebas estadísticas indicaron

que estas diferencias fueron significativas.

Estos resultados contrastan con los de la comparación inicial porque en ambas etapas se evaluaron variantes diferentes de IVNS. La primera utilizó una variante previa a la exploración, mientras que la evaluación final empleó la combinación seleccionada después del análisis de parámetros. Las semillas nuevas permitieron comprobar que su comportamiento no dependía únicamente de las ejecuciones utilizadas durante la selección.

4.4.2. Tiempo de ejecución

La Tabla 4.6 presenta los tiempos medianos de ejecución. Los valores están expresados en segundos y representan la duración completa de cada algoritmo, no el límite de tiempo asignado a una vecindad de IVNS.

Cuadro 4.6: Resultados finales de tiempo en las instancias A

Instancia	IVNS (s)	VNS (s)	Resultado
A-n33-k5	0,310	0,375	Diferencia no significativa
A-n65-k9	1,000	0,390	VNS significativamente más rápido
A-n80-k10	3,465	0,783	VNS significativamente más rápido

En A-n33-k5, la corrección de Holm produjo ($p=0,109432$). Por tanto, no se encontró una diferencia significativa en el tiempo de ejecución. En esta instancia, IVNS mejoró la calidad sin presentar una separación temporal demostrable respecto a VNS.

En A-n65-k9, VNS terminó antes en 29 de los 30 pares. La mediana de las diferencias de tiempo, calculada como IVNS menos VNS, fue de (0,625) segundos. El signo positivo indica que IVNS necesitó más tiempo.

En A-n80-k10, VNS registró el menor tiempo en los 30 pares y la mediana de las diferencias aumentó hasta (2,849) segundos.

Por tanto, la ventaja de IVNS en la calidad de las soluciones estuvo acompañada por un mayor tiempo de ejecución en A-n65-k9 y A-n80-k10. En A-n33-k5, los tiempos fueron estadísticamente comparables.

4.4.3. Estabilidad y relevancia práctica

La dispersión entre las ejecuciones permitió analizar la estabilidad de los métodos. En A-n80-k10, IVNS presentó una dispersión central inferior a la de VNS y también obtuvo la menor mediana de brecha. En A-n65-k9, la desviación estándar fue menor para IVNS. En A-n33-k5, VNS mostró una dispersión ligeramente inferior, aunque sus brechas fueron mayores.

En conjunto, **baseline-first-t1.00** produjo soluciones de mayor calidad que VNS en las tres instancias A. No obstante, esta ventaja estuvo acompañada por un mayor costo temporal en las instancias de 65 y 80 nodos estudiadas. Los resultados muestran un intercambio entre calidad y tiempo: IVNS se acercó más a la mejor solución conocida, mientras que VNS terminó antes en dos de las tres instancias.

El apartado siguiente examina el comportamiento de los métodos en **F-n135-k7**. Como solo se estudia una instancia de la familia F, los resultados posteriores se interpretan exclusivamente para ese caso.

4.5. Exploración y confirmación en F-n135-k7

El estudio incorporó **F-n135-k7** para comprobar si el comportamiento observado en la familia A se mantenía en una instancia de 135 nodos perteneciente a la familia F. En los tres casos A, el perfil base alcanzó brechas inferiores a las variantes orientadas a cadenas más complejas. Sin embargo, el piloto sobre **F-n135-k7** mostró una tendencia distinta: los ajustes de complejidad media y alta mejoraron los valores registrados por **baseline** en las combinaciones exploradas.

4.5.1. Selección exploratoria

La exploración de **F-n135-k7** se desarrolló en dos bloques. El primero comparó el ajuste base (**baseline**) y el ajuste de complejidad media (**efg-medium**), combinados con las estrategias de mejor vecino (**best**), primera mejora (**first**) y selección aleatoria (**random**). Debido al costo computacional de esta instancia, cada combinación se ejecutó cinco veces.

La segunda exploración examinó el ajuste **efg-high** mediante las estrategias **first**, **best** y **random**. Cada estrategia utilizó límites de (0,05), (0,25), (0,50), (1,00) y (1,50) segundos por vecindad. De esta relación surgieron 15 combinaciones, y cada una realizó cinco ejecuciones sobre **F-n135-k7**.

La segunda exploración examinó el ajuste **efg-high** mediante las estrategias **first**, **best** y **random**. Cada estrategia utilizó límites de (0,05), (0,25), (0,50), (1,00) y (1,50) segundos por vecindad. De esta relación surgieron 15 combinaciones, y cada combinación realizó cinco ejecuciones sobre **F-n135-k7**.

Las Tablas 4.7, 4.8 y 4.9 presentan la brecha porcentual mediana y el tiempo mediano de cada combinación. La negrita identifica la combinación que la investigación llevó a la evaluación final.

La estrategia **first** con un límite de (1,50) segundos alcanzó la menor brecha mediana de las 15 combinaciones, con (440,96), y registró un tiempo mediano de (14,07)

Cuadro 4.7: Resultados de la estrategia **first** en la segunda exploración de F-n135-k7

Límite (s)	Brecha mediana	Tiempo mediano (s)
0,05	493,98	0,80
0,25	461,62	2,59
0,50	475,99	5,22
1,00	455,25	8,78
1,50	440,96	14,07

Cuadro 4.8: Resultados de la estrategia **best** en la segunda exploración de F-n135-k7

Límite (s)	Brecha mediana	Tiempo mediano (s)
0,05	467,04	0,81
0,25	507,23	2,83
0,50	493,55	5,26
1,00	466,01	10,27
1,50	447,76	15,03

Cuadro 4.9: Resultados de la estrategia **random** en la segunda exploración de F-n135-k7

Límite (s)	Brecha mediana	Tiempo mediano (s)
0,05	484,08	0,79
0,25	474,87	2,74
0,50	489,93	4,90
1,00	472,38	9,55
1,50	489,41	15,34

segundos. Esta combinación corresponde a **efg-high-first-t1.50**.

Las tablas también muestran que un límite mayor no redujo siempre la brecha. Por ejemplo, la estrategia **random** alcanzó una brecha mediana de (474,87) con (0,25) segundos y una brecha de (489,41) con (1,50) segundos. Por tanto, el resultado dependió de la relación entre la estrategia local y el límite de tiempo.

Cada combinación realizó cinco ejecuciones. Esta cantidad permitió identificar una alternativa para la evaluación posterior, pero no permitió establecer una conclusión definitiva. Por esta razón, la investigación comparó **efg-high-first-t1.50** con VNS mediante 30 pares nuevos.

Resultado de la comparación. La Tabla 4.10 presenta los resultados.

Cuadro 4.10: Resultados confirmatorios en **F-n135-k7**.

Métrica	IVNS	VNS	Resultado
Brecha mediana	473,54	439,72	VNS significativamente mejor
Tiempo mediano	14,84 s	0,81 s	VNS significativamente más rápido

En calidad, IVNS obtuvo la brecha inferior en 4 de las 30 comparaciones pareadas, mientras que VNS lo hizo en las 26 restantes. La mediana de IVNS – VNS fue de +35,80 puntos porcentuales. Después del ajuste de Holm [13], el valor alcanzó $p = 0,000004$. La correlación biserial de rangos fue $-0,871$, lo que representa un efecto fuerte favorable a VNS [15].

En la duración, VNS completó los 30 pares en menos tiempo. La diferencia central fue de 14,02 segundos y el tamaño del efecto alcanzó $(-1,000)$.

La confirmación demostró que la variante elegida durante la exploración no superó al método de referencia. Este desenlace también evidenció que IVNS no mantiene una conducta uniforme entre familias. En los problemas A, el perfil más simple produjo las brechas inferiores; en **F-n135-k7**, los ajustes enriquecidos mostraron una tendencia favorable únicamente durante la fase exploratoria.

Por consiguiente, aunque VNS superó finalmente a IVNS en calidad y duración, el experimento aportó información sobre la influencia de las características de cada familia en el rendimiento de la metaheurística. La tendencia identificada con cinco repeticiones no se reprodujo cuando el análisis utilizó 30 puntos de partida nuevos.

A partir de los experimentos en las familias A y F, el apartado siguiente retoma los objetivos experimentales y establece conclusiones conjuntas sobre calidad, costo computacional, estabilidad y parametrización.

4.6. Discusión de las preguntas experimentales

Este apartado integra los hallazgos de las tres etapas y los relaciona con las interrogantes formuladas al inicio del estudio. El análisis reúne la calidad de las soluciones, la duración de las corridas, la estabilidad, el efecto de las estrategias locales y la influencia del generador gramatical sobre IVNS.

La primera cuestión examinó las diferencias de calidad entre VNS e IVNS. La respuesta depende del ajuste y de la instancia. En el contraste inicial, VNS alcanzó las brechas inferiores. Después de la exploración, **baseline-first-t1.00** superó al método de referencia en los tres problemas A. Por el contrario, **efg-high-first-t1.50** produjo una brecha superior en **F-n135-k7**.

La segunda interrogante abordó el tiempo de ejecución. En la confirmación sobre **A-n33-k5**, ambos algoritmos registraron duraciones comparables. VNS completó en menos tiempo **A-n65-k9**, **A-n80-k10** y **F-n135-k7**. Por tanto, la mejora de calidad alcanzada por IVNS en los casos A de 65 y 80 nodos exigió un costo computacional superior.

La tercera cuestión examinó la estabilidad y la consistencia de los resultados. Se estudió mediante la dispersión de las brechas entre ejecuciones. De manera complementaria, las victorias, derrotas y empates permitieron observar con qué frecuencia cada método obtuvo la menor brecha dentro de las comparaciones.

La Tabla 4.11 resume estas frecuencias. En esta tabla, una victoria corresponde al algoritmo que alcanzó la menor brecha porcentual dentro de cada par.

Cuadro 4.11: Consistencia de los resultados de calidad en las evaluaciones finales

Instancia	Victorias de IVNS	Victorias de VNS	Empates
A-n33-k5	19	10	1
A-n65-k9	30	0	0
A-n80-k10	30	0	0
F-n135-k7	4	26	0

En **A-n33-k5**, IVNS obtuvo la menor brecha en 19 de los 30 pares, mientras que VNS lo hizo en 10 y se produjo un empate. En **A-n65-k9** y **A-n80-k10**, IVNS obtuvo la menor brecha en los 30 pares. En cambio, sobre **F-n135-k7**, VNS venció en 26 de las 30 comparaciones.

Estas frecuencias muestran que los resultados no estuvieron determinados por una sola ejecución. No obstante, deben interpretarse junto con las medidas de dispersión y las pruebas estadísticas, ya que la cantidad de victorias no indica por sí sola la magnitud de las diferencias.

El cuarto objetivo analizó las estrategias de búsqueda local. Las variaciones entre

mejor vecino (**best**), primera mejora (**first**) y selección aleatoria (**random**) resultaron inferiores a las asociadas con los perfiles del generador. La modalidad **first** integró las combinaciones elegidas para ambas familias, pero su rendimiento dependió de los demás parámetros y del caso de prueba. En consecuencia, la estrategia local no debe interpretarse de forma independiente.

La quinta cuestión examinó el efecto de favorecer cadenas más complejas. En la familia A, **baseline** alcanzó brechas inferiores y requirió menos tiempo que **efg-medium** y **efg-high**. En **F-n135-k7**, la exploración favoreció el perfil de nivel alto; sin embargo, VNS superó posteriormente esa variante durante la confirmación. El incremento de la complejidad modifica la dinámica de IVNS, pero no garantiza un rendimiento superior.

El sexto interrogante estudió el presupuesto temporal por vecindad. En **A-n65-k9** y **A-n80-k10**, (1,00) segundo produjo soluciones de mayor calidad que los valores inferiores. La ampliación hasta (1,50) segundos no aportó una mejora estadísticamente significativa respecto a (1,00). En **F-n135-k7**, el presupuesto máximo integró la combinación exploratoria, pero no permitió superar a VNS en la fase posterior. Estos datos muestran que una duración superior favoreció la búsqueda bajo determinadas condiciones, aunque el incremento aislado del tiempo no aseguró una brecha inferior.

La séptima cuestión evaluó la estabilidad de las decisiones exploratorias. **baseline-first-t1.00** confirmó su ventaja de calidad frente a VNS sobre las tres instancias A. En cambio, **efg-high-first-t1.50** no reprodujo la tendencia inicial en **F-n135-k7**; VNS superó esta variante durante la confirmación. Esta diferencia respalda la separación entre selección exploratoria y evaluación confirmatoria.

El último interrogante abordó la transferencia de un mismo ajuste entre problemas con características distintas. Los datos no respaldan la existencia de una variante de IVNS superior en todos los casos. El perfil base resultó más adecuado para las tres instancias A, mientras que la exploración de **F-n135-k7** favoreció temporalmente un ajuste de mayor complejidad. El rendimiento depende de la interacción entre el perfil gramatical, la estrategia local, el presupuesto y las propiedades del problema. Además, el estudio incluyó una sola instancia de la familia F; por tanto, las conclusiones sobre **F-n135-k7** no se extienden a todos los miembros de esa familia.

Una vez respondidos los ocho interrogantes experimentales, la siguiente subsección presenta las limitaciones del estudio y delimita el alcance de sus conclusiones.

4.7. Limitaciones

Las conclusiones deben interpretarse dentro del alcance de la experimentación. El estudio comparó VNS e IVNS mediante tres perfiles de generación, tres estrategias de búsqueda local y los presupuestos temporales definidos en el diseño experimental. Sin

embargo, cuatro limitaciones restringen la generalización de los hallazgos.

En primer lugar, el análisis utilizó cuatro instancias del CVRP: **A-n33-k5**, **A-n65-k9** y **A-n80-k10**, pertenecientes a la familia A, además de **F-n135-k7**, correspondiente a la familia F. Por tanto, los datos describen el rendimiento de los algoritmos en estos casos de prueba y no abarcan toda la diversidad de tamaños y estructuras presentes en el CVRP.

En segundo lugar, la exploración de **F-n135-k7** ejecutó cinco corridas por combinación debido a su costo computacional. Este número permitió identificar una candidata para el contraste posterior, pero también pudo favorecer una variante cuya tendencia no se mantuvo con nuevas semillas. La confirmación mediante 30 pares proporcionó evidencia más sólida, aunque no elimina la incertidumbre que genera el tamaño reducido de la fase exploratoria.

En tercer lugar, los tiempos corresponden a la implementación en Common Lisp, la ejecución mediante SBCL, el equipo y las condiciones establecidas para los experimentos. Un cambio de lenguaje, arquitectura, optimización o procedimiento para generar y examinar vecinos modificaría potencialmente estas duraciones. En consecuencia, los valores temporales no representan medidas universales de VNS e IVNS.

En cuarto lugar, **efg-medium** y **efg-high** constituyen dos formas concretas de incrementar la presencia de los símbolos de operaciones de mayor alcance. Los hallazgos no demuestran que toda ampliación del repertorio de movimientos resulte desfavorable. Solo indican que, bajo los pesos, filtros y presupuestos estudiados, estos dos perfiles no produjeron mejoras consistentes respecto de **baseline** ni del método de referencia.

El capítulo analizó el contraste preliminar, la exploración de variantes y los experimentos confirmatorios. En la comparación inicial, VNS alcanzó brechas inferiores, lo que evidenció que una variante de IVNS sin calibración previa no garantiza soluciones de mayor calidad.

Sobre las instancias A, la exploración identificó la combinación base con primera mejora y (1,00) segundo por vecindad, denominada **baseline-first-t1.00**. Durante la confirmación, este ajuste superó a VNS en calidad en **A-n33-k5**, **A-n65-k9** y **A-n80-k10**. No obstante, la reducción de la brecha estuvo acompañada por una duración superior en los casos de 65 y 80 nodos.

En **F-n135-k7**, los perfiles **efg-medium** y **efg-high** mostraron durante la exploración una tendencia distinta de la registrada en la familia A. Este proceso eligió **efg-high-first-t1.50**, que combina el nivel alto, la estrategia de primera mejora y (1,50) segundos por vecindad. Sin embargo, VNS superó esta variante en calidad y tiempo durante el experimento confirmatorio.

En conjunto, los datos muestran que IVNS no mantiene el mismo rendimiento en todos los problemas y que ningún ajuste resultó superior bajo todas las condiciones. Su

desempeño depende de la interacción entre el perfil de generación, la estrategia local, el presupuesto temporal y las características de la instancia.

La generación dinámica de vecindades permitió que IVNS alcanzara soluciones de mayor calidad en las tres instancias A. A la vez, incrementó el costo computacional en A-n65-k9 y A-n80-k10, y produjo un resultado desfavorable en F-n135-k7. Por esta razón, la selección del ajuste requiere una exploración controlada y una confirmación posterior mediante semillas no utilizadas durante la calibración.

Conclusiones

Esta investigación evaluó experimentalmente la Búsqueda de Vecindad Infinitamente Variable (IVNS) aplicada al Problema de Enrutamiento de Vehículos con Restricción de Capacidad (CVRP). El estudio comparó su rendimiento con el de una Búsqueda de Vecindad Variable (VNS) tradicional y examinó la influencia de la construcción gramatical de criterios sobre la calidad de las soluciones, la duración de las corridas y su estabilidad.

El trabajo adaptó una plataforma experimental para ejecutar ambos algoritmos bajo condiciones controladas, utilizar puntos de partida comunes, registrar la información de cada corrida y producir ficheros de resumen y traza. La infraestructura también incorporó procedimientos de limpieza y validación, estadísticas descriptivas, contrastes no paramétricos y generación de tablas y figuras.

La experimentación comprendió tres fases: comparación inicial, exploración y confirmación. La segunda permitió elegir variantes de IVNS, mientras que la tercera utilizó semillas nuevas para comprobar su comportamiento. En los contrastes pareados, la firma, el costo y la procedencia de la solución inicial confirmaron que VNS e IVNS comenzaron desde el mismo estado; por tanto, la correspondencia no dependió exclusivamente de la semilla.

El contraste preliminar mostró que una variante de IVNS sin calibración produjo brechas superiores a las de VNS en A-n33-k5, A-n65-k9 y A-n80-k10. Este hallazgo demuestra que la generación dinámica de entornos no garantiza por sí sola un rendimiento superior y que la eficacia del método depende de su ajuste.

El análisis interno reveló que el perfil base (**baseline**) alcanzó brechas inferiores y requirió menos duración que **efg-medium** y **efg-high**. Bajo las condiciones estudiadas, favorecer cadenas más extensas o acciones de mayor alcance incrementó el costo de evaluación sin aportar mejoras consistentes en la función objetivo.

La exploración sobre la familia A identificó **baseline-first-t1.00**, combinación formada por el perfil base, la estrategia de primera mejora y un tiempo de (1,00) segundos por vecindad. En A-n65-k9 y A-n80-k10, esta configuración produjo soluciones de mayor calidad que aquellas con menos tiempo por vecindad. El aumento hasta (1,50) segundos no generó una reducción estadísticamente significativa de la brecha respecto a (1,00), lo

que evidencia un rendimiento decreciente del tiempo adicional.

Durante la confirmación, IVNS superó a VNS en calidad en las tres instancias A. En A-n65-k9 y A-n80-k10, la propuesta alcanzó la brecha inferior en los 30 pares. En A-n33-k5, obtuvo la ventaja en 19 comparaciones, VNS en 10 y el par restante terminó en empate. Estas frecuencias muestran que la diferencia no dependió de ejecuciones aisladas.

La reducción de la brecha exigió, no obstante, un costo computacional adicional. En A-n33-k5, el análisis no detectó una diferencia temporal significativa. En cambio, VNS registró tiempos inferiores en A-n65-k9 y A-n80-k10. En estos dos casos, IVNS estableció un compromiso entre mayor calidad y mayor tiempo de ejecución.

El estudio de F-n135-k7 presentó una tendencia distinta. La exploración eligió *efg-high-first-t1.50*, integrada por el perfil de complejidad alta, primera mejora y (1,50) segundos por entorno. Sin embargo, VNS superó esa variante durante la confirmación: alcanzó soluciones de mayor calidad en 26 de los 30 pares y completó todas las comparaciones en menos tiempo.

La exploración de F-n135-k7 utilizó cinco corridas por combinación, mientras que el experimento confirmatorio empleó 30 pares nuevos. La menor cantidad de observaciones de la primera fase limitó la solidez de la selección y explica por qué la tendencia inicial requería una comprobación independiente. Este resultado respalda la separación entre calibración y confirmación.

Los datos no justifican afirmar que IVNS resulta siempre superior o inferior a VNS. En la familia A, el perfil *baseline* mejoró la calidad, aunque aumentó la duración en los casos de 65 y 80 nodos. En F-n135-k7, VNS obtuvo ventajas en ambas métricas frente a una combinación orientada a cadenas de mayor nivel estructural. Como el estudio solo evaluó una instancia de la familia F, esta conclusión corresponde exclusivamente a dicho problema.

La principal conclusión radica en que el potencial de IVNS no depende únicamente del número de criterios disponibles. Su rendimiento exige controlar las cadenas derivadas, comprobar su validez y aplicabilidad, limitar el costo de examinar los vecinos y favorecer operaciones capaces de reducir la función objetivo. En los experimentos realizados, una generación selectiva proporcionó mejores perspectivas que el incremento directo de la longitud o la complejidad.

Los objetivos propuestos se cumplieron. La investigación estudió los fundamentos del CVRP, VNS e IVNS; examinó la representación gramatical de los criterios; adaptó una plataforma reproducible; y desarrolló comparaciones respaldadas por estadísticas descriptivas, pruebas no paramétricas y tamaños del efecto. Asimismo, identificó una variante competitiva para las tres instancias A y delimitó las fortalezas y restricciones observadas.

Estos hallazgos ofrecen una base experimental para continuar el desarrollo de IVNS. Las investigaciones posteriores podrán estudiar nuevos mecanismos para orientar la gramática, reducir el costo de las expresiones complejas y adaptar la selección de criterios a las características de cada instancia.

Recomendaciones

Los resultados indican que el desarrollo futuro de IVNS debe orientarse hacia una generación más selectiva de cadenas. Los perfiles demostraron que el incremento directo de la complejidad no garantiza una reducción del costo de las soluciones y, además, eleva la duración de las corridas. Por tanto, el generador debe priorizar criterios con una frecuencia elevada de vecinos factibles, mejoras aceptadas y evaluaciones completadas dentro del presupuesto.

Resulta prioritario fortalecer las validaciones sintácticas y semánticas aplicadas antes de examinar cada expresión. Las comprobaciones actuales descartan estructuras que incumplen las reglas definidas, pero el algoritmo todavía dedica recursos a criterios que no generan vecinos factibles o no reducen la función objetivo. Una ampliación del validador debe considerar la estructura de las rutas, la capacidad restante de los vehículos, la longitud de la secuencia y el costo estimado de las operaciones antes de ejecutar el criterio.

Las investigaciones posteriores deben incorporar mecanismos adaptativos. En lugar de conservar probabilidades fijas durante toda la corrida, el algoritmo debería registrar la frecuencia con que cada símbolo, patrón o secuencia produce una mejora válida. A partir de esa información, el generador aumentaría el peso de las estructuras exitosas y reduciría temporalmente el de aquellas asociadas con vecinos inválidos, agotamiento del presupuesto o ausencia reiterada de avances.

La política de reinicio del historial después de una mejora debe mantenerse, pero conviene complementarla con dos niveles de almacenamiento. El primero conservaría la lista temporal destinada a evitar repeticiones dentro de una fase de búsqueda. El segundo reuniría un historial global de utilidad con la frecuencia de éxito, el costo medio de evaluación y las causas de rechazo de cada criterio. Esta separación favorecería la diversificación sin eliminar el conocimiento acumulado durante la corrida.

El diseño experimental debe ampliarse mediante una muestra estratificada de la biblioteca de instancias CVRPLIB. El protocolo debe seleccionar casos según la familia, el número de nodos, la distribución espacial y la relación entre demanda y capacidad. También debe incluir más de una instancia por estrato cuando los recursos computacionales lo permitan. La diferencia entre la familia A y F-n135-k7 impide

extender automáticamente las conclusiones de un único caso a todos los problemas con características semejantes.

La exploración de los casos con mayor costo computacional necesita un número de repeticiones definido antes del experimento o un procedimiento secuencial. Las cinco ejecuciones por combinación redujo la duración total, pero no identificó una variante que conservara su tendencia durante la confirmación. Un esquema secuencial debería eliminar los ajustes cuyo rendimiento quede por debajo de un umbral previamente fijado y asignar observaciones adicionales a las candidatas restantes.

Los límites temporales por vecindad también deben adaptarse a las características del problema y del criterio. En las instancias A, (1,00) segundo alcanzó una calidad semejante a la obtenida con (1,50), mientras que (0,20) y (0,50) segundos resultaron insuficientes en determinados casos de 65 y 80 nodos. Un mecanismo adaptativo debería calcular el presupuesto a partir de la dimensión de la instancia, la longitud de la cadena y el costo histórico del entorno correspondiente.

Los estudios posteriores deben incorporar comparaciones bajo presupuestos equivalentes. Además de registrar el tiempo como resultado, conviene contrastar VNS e IVNS mediante una duración total común, el mismo número de evaluaciones de vecinos o una cantidad equivalente de llamadas a la función objetivo. Este diseño permitiría distinguir la eficiencia algorítmica del costo derivado de la implementación y del intérprete gramatical.

Otra línea de investigación consiste en desarrollar estrategias híbridas. Una alternativa usaría las vecindades fijas de VNS durante el progreso normal e incorporaría criterios generados por IVNS después de alcanzar un umbral de estancamiento. Otra posibilidad emplearía el generador fuera de línea para identificar expresiones con alta frecuencia de mejoras factibles y añadirlas posteriormente al repertorio finito de VNS.

Las estrategias de búsqueda local requieren un análisis conjunto con el perfil gramatical. Mejor vecino (**best**), primera mejora (**first**) y selección aleatoria (**random**) no deben estudiarse como factores independientes, porque su rendimiento varía según la complejidad y el costo del criterio. Las investigaciones futuras deberían seleccionar la modalidad local a partir de la longitud de la cadena, el número estimado de candidatos y el progreso observado en las iteraciones anteriores.

La implementación también necesita optimizar la interpretación y ejecución de las expresiones. La compilación de criterios, el almacenamiento en caché de secuencias repetidas, la evaluación incremental del costo y el uso de estructuras de datos especializadas reducirían la diferencia temporal observada en A-n65-k9, A-n80-k10 y F-n135-k7. Asimismo, el intérprete debería interrumpir una operación tan pronto detecte que esta resulta incapaz de producir un vecino factible.

La plataforma experimental debe conservar y ampliar las prácticas de reprodu-

cibilidad. El sistema debe mantener separados los bloques iniciales, exploratorios y confirmatorios; automatizar los informes; registrar las versiones del código, SBCL y Python; guardar los parámetros de cada corrida; y conservar las firmas, costos y procedencias de las soluciones iniciales. Estas comprobaciones evitan aplicar pruebas pareadas cuando las observaciones no comparten un punto de partida verificable.

Finalmente, la generación gramatical puede extenderse a otras variantes del VRP y a problemas adicionales de optimización combinatoria. Cada aplicación exigirá definir sus operaciones elementales, restricciones semánticas, reglas de factibilidad y medidas de costo. La experiencia obtenida muestra que la gramática debe integrarse con mecanismos de validación, adaptación y control computacional, en lugar de funcionar únicamente como una fuente de diversidad.

Bibliografía

- [1] Johnson, D. S. (2002). A Theoretician's Guide to the Experimental Analysis of Algorithms. En M. H. Goldwasser, D. S. Johnson y C. C. McGeoch (Eds.), *Data Structures, Near Neighbor Searches, and Methodology: Fifth and Sixth DIMACS Implementation Challenges*, DIMACS Series in Discrete Mathematics and Theoretical Computer Science, vol. 59, pp. 215–250. American Mathematical Society.
- [2] Montgomery, D. C. (2017). *Design and Analysis of Experiments*. 9th ed. John Wiley & Sons.
- [3] Demšar, J. (2006). Statistical Comparisons of Classifiers over Multiple Data Sets. *Journal of Machine Learning Research*, 7, 1–30.
- [4] Pérez Mosquera, C. (2017). *Primeras aproximaciones a la Búsqueda de Vecindad Infinitamente Variable*. Trabajo de Diploma en opción al título de Licenciado en Ciencia de la Computación. Facultad de Matemática y Computación, Universidad de La Habana, La Habana, Cuba.
- [5] Uchoa, E., Pecin, D., Pessoa, A., Poggi, M., Vidal, T. y Subramanian, A. (2017). New Benchmark Instances for the Capacitated Vehicle Routing Problem. *European Journal of Operational Research*, 257(3), 845–858. <https://doi.org/10.1016/j.ejor.2016.08.012>.
- [6] Augerat, P., Belenguer, J. M., Benavent, E., Corberán, A., Naddef, D. y Rinaldi, G. (1995). *Computational Results with a Branch-and-Cut Code for the Capacitated Vehicle Routing Problem*. Informe técnico INPG-RR-949-M, Université Joseph Fourier, Grenoble, Francia.
- [7] Fisher, M. L. (1994). Optimal Solution of Vehicle Routing Problems Using Minimum K -Trees. *Operations Research*, 42(4), 626–642. <https://doi.org/10.1287/opre.42.4.626>.
- [8] Steele, G. L., Jr. (1990). *Common Lisp: The Language*. 2nd ed. Digital Press. <https://www.cs.cmu.edu/Groups/AI/html/cltl/cltl2.html>.

-
- [9] The SBCL Team. (2026). *Steel Bank Common Lisp User Manual*. <https://www.sbcl.org/manual/>. Consulta: 20 de junio de 2026.
 - [10] Python Software Foundation. (2026). *Python 3.12 Documentation*. <https://docs.python.org/3.12/>. Consulta: 20 de junio de 2026.
 - [11] Barr, R. S., Golden, B. L., Kelly, J. P., Resende, M. G. C. y Stewart, W. R. (1995). Designing and Reporting on Computational Experiments with Heuristic Methods. *Journal of Heuristics*, 1(1), 9–32. <https://doi.org/10.1007/BF02430363>.
 - [12] Hollander, M., Wolfe, D. A. y Chicken, E. (2014). *Nonparametric Statistical Methods*. 3rd ed. John Wiley & Sons. <https://doi.org/10.1002/9781119196037>.
 - [13] Holm, S. (1979). A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics*, 6(2), 65–70.
 - [14] Wilcoxon, F. (1945). Individual Comparisons by Ranking Methods. *Biometrics Bulletin*, 1(6), 80–83. <https://doi.org/10.2307/3001968>.
 - [15] Kerby, D. S. (2014). The Simple Difference Formula: An Approach to Teaching Nonparametric Correlation. *Comprehensive Psychology*, 3, Article 1. <https://doi.org/10.2466/11.IT.3.1>.
 - [16] Mann, H. B. y Whitney, D. R. (1947). On a Test of Whether One of Two Random Variables Is Stochastically Larger than the Other. *The Annals of Mathematical Statistics*, 18(1), 50–60. <https://doi.org/10.1214/aoms/1177730491>.
 - [17] Kruskal, W. H. y Wallis, W. A. (1952). Use of Ranks in One-Criterion Variance Analysis. *Journal of the American Statistical Association*, 47(260), 583–621. <https://doi.org/10.1080/01621459.1952.10483441>.
 - [18] Cliff, N. (1993). Dominance Statistics: Ordinal Analyses to Answer Ordinal Questions. *Psychological Bulletin*, 114(3), 494–509. <https://doi.org/10.1037/0033-2909.114.3.494>.
 - [19] Dantzig, G. B. y Ramser, J. H. (1959). The Truck Dispatching Problem. *Management Science*, 6(1), 80–91. <https://doi.org/10.1287/mnsc.6.1.80>.
 - [20] Toth, P. y Vigo, D. (Eds.). (2014). *Vehicle Routing: Problems, Methods, and Applications*. 2nd ed. MOS-SIAM Series on Optimization, vol. 18. Society for Industrial and Applied Mathematics. <https://doi.org/10.1137/1.9781611973594>.
 - [21] Laporte, G. (2009). Fifty Years of Vehicle Routing. *Transportation Science*, 43(4), 408–416. <https://doi.org/10.1287/trsc.1090.0301>.

-
- [22] Gendreau, M. y Potvin, J.-Y. (Eds.). (2010). *Handbook of Metaheuristics*. 2nd ed. International Series in Operations Research & Management Science, vol. 146. Springer. <https://doi.org/10.1007/978-1-4419-1665-5>.
- [23] Blum, C. y Roli, A. (2003). Metaheuristics in Combinatorial Optimization: Overview and Conceptual Comparison. *ACM Computing Surveys*, 35(3), 268–308. <https://doi.org/10.1145/937503.937505>.
- [24] Mladenović, N. y Hansen, P. (1997). Variable Neighborhood Search. *Computers & Operations Research*, 24(11), 1097–1100. [https://doi.org/10.1016/S0305-0548\(97\)00031-2](https://doi.org/10.1016/S0305-0548(97)00031-2).
- [25] Hansen, P., Mladenović, N. y Moreno Pérez, J. A. (2010). Variable Neighbourhood Search: Methods and Applications. *Annals of Operations Research*, 175(1), 367–407. <https://doi.org/10.1007/s10479-009-0657-6>.
- [26] Hopcroft, J. E., Motwani, R. y Ullman, J. D. (2006). *Introduction to Automata Theory, Languages, and Computation*. 3rd ed. Pearson/Addison-Wesley.